SENG406
Assignment 2

Threat Modelling and Attack Surface Evaluation

Group 15

Hannah Botting

Sophia Copley

Alex McLauchlan

William Tingey

AI Usage Declaration

Generative Artificial Intelligence tools, such as ChatGPT and Claude, were used in the following ways to support the completion of this assignment:

- Reviewing the report to identify grammatical errors and sections where placeholders had been left unfilled.
- Explaining the mechanisms behind various threat mitigation methods where the underlying nuances were not fully covered in the lecture notes, recommended readings, or search engine results.

GitHub Copilot was used to generate a script that de-duplicated the reference numbers in the Threat Dragon reports when the two models were merged into one.

Actors of the system, roles and responsibilities

An exhaustive list of the actors, roles, and responsibilities in the system are detailed below in Table 1.

Table 1: Actors with roles and responsibilities.

ID	Actor	Description
A1	Citizen	Registers, verifies their email, submits reports with images, views and edits their own open reports, and generates share links for them
A2	Share link recipient	Views a single report read-only using a token received outside the system. Never authenticates and is unknown to the system
A3	Police officer	Searches and views all reports, emails owners requesting more information, and closes reports.
A4	System Administrator	Operates the server, proxy, firewall and database.

System Dependencies: Provided and Assumed

A list of the dependencies of the system is detailed below in Table 2.

Table 2: System dependencies – provided and assumed.

ID	Dependency	Description
D1	Server operating system	The host the application runs on, including its patching and access controls
D2	TLS certificate	Issued by a certificate authority, required for the HTTPS-only constraint
D3	Reverse proxy	Handles incoming requests before they reach the application
D4	Firewall	Restricts which traffic can reach the server
D5	Web application framework	Application is built on a web framework and its language runtime, which handle things like URL routing and page rendering.
D6	Database	Stores the accounts, reports and share tokens, reachable only from the application
D7	Password hashing library	Used to store account passwords (assumed third-party)
D8	Random number generator	Produces the verification codes and share tokens (assumed third-party)
D9	Image handling library	Processes uploaded report images and writes them to storage
D10	Email dispatcher	Sends the verification, information request and closure emails
D11	Web browser	Enforces cookie handling and certificate checks on the user's side

Constraints and Non-Functional Requirements: Provided and Assumed

The constraints and non-functional requirements are detailed below in Table 3.

Table 3: Constraints and non-functional requirements.

ID	Constraint	Description
C1	Public Availability	The web application is available publicly on a website, accessible from both desktop and mobile devices.
C2	HTTPS Encrypted Traffic	The web application is only available via HTTPS, and the data is encrypted in transit.
C3	Co-located System and Database	The system is deployed on premises, and the data is stored in a database on premises too.
C4	Firewall & Reverse Proxy	The web application is deployed on a server protected by a firewall, and a reverse proxy is used to handle requests.
C5	Internet Inaccessible Database	The database is not directly accessible from the internet, and only the web application can access it.
C6	Firewall & Proxy Rate Limiting	Both the firewall and reverse proxy have per-IP rate limiting implemented.
C6	Multiple Report Share Links	A user is assumed to be able to create multiple share links for each report.
C7	Logging	- API endpoints that are hit are logged with each request's endpoint, method, timestamp, source IP, and response status. - Application (business logic) relevant events are logged: authentication including success or failure status, authorisation failures, important events such as a report being created or closed, and modifications to data. Each log entry includes timestamp, actor id (or "anonymous" if unauthenticated), action performed, target resource, and outcome. - Application and API logs are stored in a separate database from the application database. - The system relies on the database management system (DBMS) to log database activity. These are stored in files on the DBMS server
C8	Login page for Citizens and Police	The login page for citizens and police officers are the same.

Identified Trust Levels

An exhaustive list of the trust levels is detailed below in Table 4.

Table 4: Trust levels of the system.

ID	Name	Description
T0	Unauthenticated user	Any internet user who has not logged in. Reaches only public pages.
T1	Registered user, email not verified	An account exists, but the system refuses to login until the email is verified.
T2	Registered user, email verified but not logged in	A verified account exists but is not logged in yet. Has the same privileges as T0
T3	User with valid share token	Unauthenticated, but the token grants read access to the one open report it refers to. Access is revoked by the system when the report status changes to closed.
T4	Logged in citizen	Can create reports but does not have access to any reports.
T5	Report owner	A citizen acting on reports they created. They can edit open reports and view closed reports.
T6	Logged in police officer	Can search and view every report, email any owner, and close any report.
T7	Web application process	The identity the application runs as, holding the database and mail credentials.
T8	Database process	Direct access to stored data, bypassing every application check.
T9	System administrator	Full control of the host, proxy, logs and backups.

Entry Points

An exhaustive list of the entry points in the system is detailed below in Table 5.

Table 5: Entry points with descriptions and trust levels.

ID	Name	Description	Trust Levels
E1	Registration page and submission	The form where a new citizen enters an email and password, where a new account is created and a verification link is sent.	T0
E2	Email verification link	The link sent to the user's email, that upon clicking, sends a code to the system and marks the account as verified.	T1
E3	Login Page	The page where users enter their email and password, and then check if they log into their account	T0 - T4
E4	Logout function	The action which signs a logged in user out	T4, T5

E5	Report list page	The page listing the reports which a citizen owns, showing the status of each report	T4
E6	Create report page	The form where a citizen enters the date, time, location, description and images of a new report	T4
E7	View and edit report page	The page where a report owner views and edits a report from	T5
E8	Share link creation	The action which creates the secret link for a report	T5
E9	Share link page	The public address that receives a secret token, if the token is valid, it shows the matching report	T3
E10	Report image retrieval (Assuming separate storage)	The address that stores the images for each report	T3, T5, T6
E11	Report search page	The form where an officer searches reports by date, location, status or keyword	T6
E12	Officer report details page	The page where an officer views the full details of a report	T6
E13	Request more info form	The form where an officer provides the extra details required, and the system emails this to the report owner	T6
E14	Close report action	The action which closes a report and notifies its owner	T6
E15	HTTPS port	The secure port which the website is published on	T0
E16	Reverse proxy	The proxy that handles incoming traffic before it reaches the application	
E17	Firewall configuration	The rules controlling which traffic is allowed to reach the server	T2
E18	Administrative host access	Remote maintenance access to the server. Reaches the filesystem, logs and backups	T2
E19	Database listener	The database port on the internal network	T7

Exit points

An exhaustive list of the entry points in the system is detailed below in Table 6.

Table 6: Exit points with descriptions and trust levels.

ID	Name	Description	Trust Levels
X1	Citizen report responses	The report list and report details sent back to the citizen who owns them	T4, T5

X2	Share link response	The report details sent to anyone with the secret share link	T3
X3	Officer search and detail responses	The search results and report details sent to an officer	T6
X4	Report image responses	The image files sent back to whoever is viewing a report	T3, T5, T6
X5	Verification email	The email containing the verification link	T1
X6	Request more info email	The email an officer sends a report owner asking for more information	T4, T5
X7	Closure notification email	The email which gets sent to a report owner when a report gets closed	T4, T5
X8	Registration and login responses	The confirmations, redirections and error messages reveal whether an account exists and is verified	T0, T1, T4
X9	System logs	The log files written by the application, database and server, which reveals information about application state	T2, T7

Identified Assets

An exhaustive list of the assets in the system is detailed below in Table 7.

Table 7: Assets in the system with descriptions and trust levels.

ID	Asset	Description	Trust levels
AS1	Citizen credentials	Email address and password used to log in, stored in the database.	T7
AS2	Verification codes	Codes which grant account activation	T7
AS3	Session identifiers	Issued at login, identifier which user is logged in to the system	T4, T5, T7
AS4	Officer credentials	Username and password	T7
AS5	Report content	Date, time, location and description of an incident. Date, time, location and description of an incident	T5, T6, T7, T8
AS6	Report images	Uploaded photographs related to an incident	T5, T6, T7, T8
AS7	Share tokens	Secret values granting read access to a report without logging in	T3, T5, T7, T8
AS8	Availability	The application should always be available to citizens and officers	T7, T9
AS9	System credentials	Database and email credentials and the session signing key, held in the application configuration	T7, T9

AS10	Server as a computing resource	Processing, storage and network capacity that could be consumed or misused	T7, T9
AS11	Backups	Copies of the database and stored images on separate media	T9
AS12	System logs	Server, application and proxy logs, which record requested addresses.	T7, T9
AS13	Application libraries	The framework, image handling, hashing and token generation libraries	T7, T9
AS14	Database management system	Stores every account, report and token	T8, T9
AS15	Email dispatcher	Delivers the system's mail	T7, T9
AS16	Source code	The deployed code on the server, which reveals how authorisation, tokens and uploads are implemented	T9
AS17	Police information request payload	The subject line and body of the 'request information' emails submitted by Police Officers	T6, T7
AS18	Database schematic	Database structure, indexes, and triggers	T7, T8, T9
AS19	TLS Server Certificates	Private keys used to secure HTTPS traffic	T7, T9
AS20	Server Storage	Storage volumes on the host server such as those used to store report images	T7, T8, T9
AS21	Data integrity	The integrity of the stored data to remain in an untampered state (as a business asset)	T5, T6, T7, T8, T9
AS22	Trust	Trust of the public in the security and confidentiality of SecureReports	T0-T9
AS23	Database ACID Guarantees	The guarantee of data transactions to the database to remain atomic, consistent, isolated, and durable.	T8, T9

Proposed Improvements

Analysis and Threat Modelling

Multi-factor Authentication

Logging in to the system only requires a citizen's email address and password, and for police it requires a username and password. There are many ways that a bad actor can steal credentials of a user of this system and allowing a bad actor can log in to the system pretending to be someone else. This is especially problematic if the stolen credentials are the police's credentials, as this grants a bad actor elevated privileges of

viewing all reports, requesting citizens to provide more details, and closing all reports so they can't be edited again. Therefore, it is important that the system takes additional measures to verify the user is who they say they are. The system needs to verify users with something you know, something you have, and something you are.

A proposed change would be to add multiple factor authentication (MFA). There are many ways that MFA can be done: SMS verification codes, fingerprint or facial recognition, authenticator apps, and passkeys. Pass-keys are the most secure. They work by storing a private key securely on dedicated security hardware of your device and requiring a PIN or biometric recognition to unlock it, replacing the username and password process entirely. However, some older computers and phones may lack the security hardware to make this work and changing the log in flow to replace usernames and passwords could confuse existing users. Therefore, to maximise both simplicity for users and security, we propose the use of an authenticator app such as Microsoft or Google Authenticator, specifically the flow that involves push notifications and number matching.

When a user registers, they will need to scan a QR code on their existing authenticator app on their device to add an account. When a user logs in with their username and password (something you know), instead of being able to automatically access the functionality, the website will show a code, and a push notification will appear on the user's device (something you have). The app requires either face ID or a PIN (something you are) to authenticate, and then the correct code entered in to be fully authenticated and access the functionality. For simplicity, assume that a user already has an authenticator app downloaded and set up on their mobile device.

The security of an authenticator app is not perfect. It is susceptible to phishing attacks where a bad actor replaces the login page with a malicious site, passes your credentials to the real site, triggering 2FA. If the malicious site mimics this process, users are tricked into will type the code into the authenticator app and authenticate the bad actor into the application. However, it is more secure than other code-based MFA methods like an SMS code which rely on cellular networks that can be sniffed and are vulnerable to SIM-swapping. The number matching flow is an improvement on regular one-time passcodes, because the code is associated with a specific log in attempt, rather than being a general code that resets every 30s, so this version should be used on the app.

The ultimate reason this is the most appropriate measure is familiarity and convenience. Most users will be set up with an authenticator app, meaning that the logging in process will be familiar and easy for users to adapt to. In addition, the website does not have to implement much infrastructure themselves to get this to work. Relying on an external dependency to manage MFA does introduce risk of a vulnerabilities that are out of this company's control, however, Google and Microsoft are trusted entities,

and it is likely if it was implemented in-house it would be expensive, and more vulnerable.

Lack of Verification Code Expiry

Registration verification codes do not expire. If an attacker registers an account using an email address they do not own, and that email is later compromised, the original verification code is still valid and can be used to activate an account registered under someone else's address.

With no expiry and no rate limit, an attacker can guess codes indefinitely, staying below any volume that would appear suspicious. Expiry bounds that window, although the primary defences to prevent guessing verification codes are rate limiting and code entropy.

The proposed change is for the Citizen class to gain a field, verification_code_expiry, which is a time, set to 10 minutes after the initial registration of the email. If a citizen follows the verification link after that time, verification is refused and they are shown a message explaining that they must request a new link.

A button reading "Didn't get your verification code?" is permanently visible on the login page, leading to a form where a user enters their email address to receive a new link. This button is shown unconditionally, so its presence does not reveal whether an account exists.

By having a 10-minute expiry on codes, the likelihood that an attacker will be able to verify an account which is not theirs is vastly reduced but is not eliminated. If the attacker has access to a compromised email, there is no way that the system can stop them from verifying this email. The specified system also provides no password reset and no way to change a registered address, so a user who loses access to their mailbox has no recovery path regardless of this change. That limitation is out of scope for this proposal but bounds what expiry can achieve.

One trade-off of this approach is the risk of poor user experience when a citizen attempts to verify their account after the code has expired, requiring them to restart the verification process and request a new code. This also increases load on the email dispatcher, as each expired attempt generates an additional outbound email that would not have been necessary had the original code still been valid. By minimising the attack surface through short-lived codes, the usability of SecureReports may be marginally affected. However, this is a worthwhile compromise compared to the alternative of no expiry at all.

Expiry and revocation of secret links

The current design allows citizens to generate secret links that give unauthenticated access to their report. Share tokens have no lifetime and no way to withdraw them, and anyone with the link can view the open case. This means that possession of the link is the only authorisation mechanism and can effectively provide permanent read access to the report. On top of this, if the owner no longer wants the incident to be viewable by other people, or the token in leaked, they have no way to revoke this.

This presents an aspect of the system which is unsecure by design. Secret links may be intentionally shared with a limited set of people, but the owner has no way to regain control once the link has been distributed. For example, a recipient could forward the link to other people, post it publicly, or have the link exposed through another service. Any person who subsequently obtains the token can access the report without authentication. This is particularly significant because reports contain sensitive information such as the date, time, location and description of an incident, as well as potentially sensitive images.

We propose a potential improvement to this aspect of the system as introducing both expiration and revocation to share links. When a share link is created, the system should associate it with an expiration time. The owner should also be able to revoke an existing token from their report management interface, immediately preventing access through that link. The system should verify that a token is valid, has not expired, and has not been revoked, before returning the report. Revoked or expired tokens should be rejected by the share-link endpoint, while the report itself remains available to its owner.

This proposed change carries a similar trade-off to adding expiry to verification codes. A legitimate recipient who attempts to access the link after it has expired will be denied and must ask the report owner to generate a new one. This introduces friction for recipients with a genuine ongoing need to view the report, particularly if the owner is slow to respond or is unaware the link has expired. However, this is a reasonable compromise given the sensitivity of report content and the large risk of leaving reports permanently exposed.

Overall, expiration and revocation significantly reduce the lifetime and persistence of leaked share tokens. They do not prevent a recipient from copying information while the token is valid, but they provide the report owner with meaningful control over how long the report remains accessible through an unauthenticated link.

The “request more info” email is unauthenticated, unrecorded and untracked

When a report lacks detail, a police officer can request more information by filling in a free text subject and body, which the system then emails to the report's owner. When a

message is sent, it arrives from the police service's own domain with valid credentials, and contains whatever content was provided in the free text editor. If a Bad Actor gains access to a compromised officer account, they can use search_reports() to obtain the email addresses of every citizen in the system and then use the request form to deliver phishing emails containing malicious content that are indistinguishable from genuine police officer requesting more details. Because the system does not store the content of these emails, or which officer sent them, no post incident tracking can occur. A secondary issue which this system is that the specific request from a police officer is sent in the email directly to the user's email address, which exposes potentially sensitive info about the status of reports outside the systems trust level boundaries. This means the system cannot track if this information has been viewed and by who.

The proposed change is for a new ReportMessage class to be introduced, with fields report, officer, body, created_at and read_at. An officer creates the request in the application, and it is written to the database rather than sent. The outbound email is reduced to a fixed template containing no officer supplied content stating only that there is an update on one of the citizen's reports and that they should log in to view it. The citizen reads the message on the report page and responds by editing the report as usual. Because the message must be stored to be read in the application, a record of which officer wrote what, and when, exists because of the change rather than as a separate mechanism added alongside it.

Removing officer authored content from outbound email eliminates obvious phishing it does not eliminate phishing. An attacker can still send mail claiming to originate from the service. However, users are less likely to fall for a phishing email that comes from an illegitimate domain. This improvement significantly mitigates phishing attacks that users are very likely to fall for; an attacker with a compromised officer account is restricted from sending whatever malicious content they want directly to a user's inbox with SecureReports' own infrastructure.

The new improvement means no links will be sent to the user and info will instead be stored in a separate datastore. This has the caveat that it increases the amount of personal data the system retains that could be vulnerable to exploitation. Despite this risk, storing the extra data is worth it because database systems are simpler to secure than a human's behaviour, and avoiding sending links to users will make them trust the system more, and they will be more likely to update their reports as requested.

Finally, the change removes the delivery channel but not the target list. A compromised officer account still exposes every report and every registered email address in the system which can be used to create convincing phishing attacks. This limits the improvement significantly, and suggests that other measures, such as two factor authentication or applying principle of least privilege to officer accounts (perhaps so

that they can’t see the full email of citizen who owns the report) are necessary to protect from compromising these accounts in the first place.

Police and Citizens

Owner: Sophia Copley
Reviewer:
Contributors:
report.releasedAt: Wed Aug 12 2026

Executive Summary

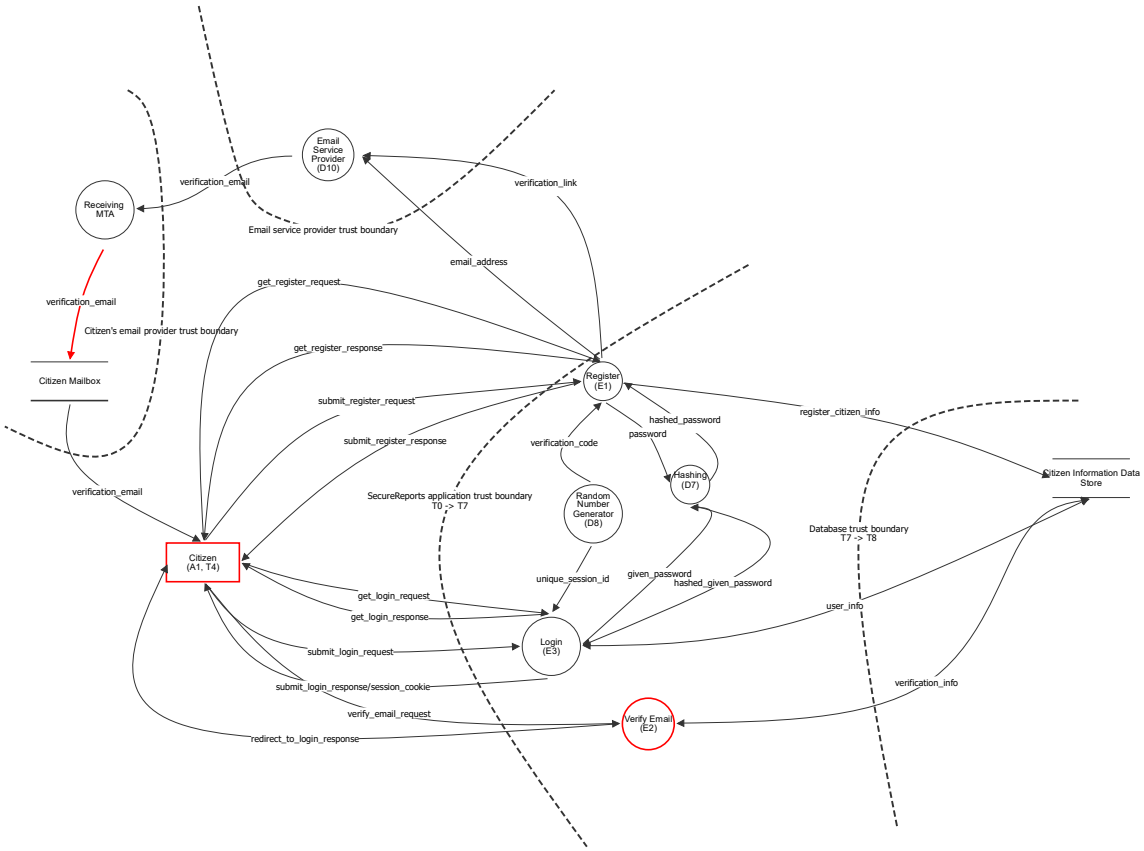
High level system description

Not provided

Summary

Total Threats	192
Total Mitigated	181
Total Accepted	4
Total Open	7
Open / Critical Severity	0
Open / High Severity	3
Open / Medium Severity	1
Open / Low Severity	0
Open / TBD Severity	3

Citizen Login and Register



Citizen Login and Register

Citizen (A1, T4) (Actor)

Description: A Citizen is a user of the system who submits reports to be read and responded to by police officers

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Weak password	Spoofing	TBD	Mitigated		Passwords can be guessed if they are too weak, or have been reused on another application where a data breach occurred.	- Enforce strong password rules such as 10 characters and at least one of capital letter, number, special character, or pass-phrases - Limit the number of times a user can attempt to login (so that bad actors have less chances at guessing correctly)
2	Citizen doesn't own email address	Spoofing	TBD	Mitigated		The user registers with an email they don't own.	- Use email verification code after registration.
3	Bad actor logs in using stolen credentials	Spoofing	TBD	Open		Bad actors finds a user's email and password (through measures like phishing emails etc) and attempts to use it to log in.	
4	User's session cookie is stolen	Spoofing	TBD	Mitigated		A bad actor obtains a session cookie belonging to a legitimate authenticated user, allowing them to impersonate a citizen by without knowing their password.	- Short session expiry or cookie-token lifetime - Session id rotation after privilege changes or re-authentication - Require re-authentication for sensitive actions - Use session binding to reject or flag requests where the session is used from a different context

Citizen Information Data Store (Store)

Description: Database storing citizen registration details: email address, hashed password, verification code and email verification status.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	Exfiltrate data	Information disclosure	TBD	Mitigated		An unregistered or unprivileged process or user tries to get access to the datastore.	- Use strong credentials for database access stored in secured password vaults. - Credentials are passed at runtime from environment variables for most DB owners / users with least privilege principle. - Disable default DB user. - Separate between read and read/write permissions. - Monitor for unusual activity, e.g., bulk data accessed, unusual number of simultaneous requests or connections. NOTE: Copied straight from example

Number	Title	Type	Severity	Status	Score	Description	Mitigations
6	DB is accessed and tampered with	Tampering	TBD	Mitigated		A bad actor or process tamper the datastore directly.	- Allow full access to datastore to only a restricted set of personnel. - Store strong credentials in password vaults. - Monitor for and log all direct command line accesses to the datastore. - Keep regular backups in a separate location.
7	DB is unreachable	Denial of service	TBD	Mitigated		A bad actor floods the datastore with requests and overloads the database connection pool, preventing access to the database.	- Monitor suspicious requests and disallow requests from abusing local IPs or ban abusing user. - Allow for replication and deployment with limited downtime when the load increases slightly. - Use a pool of connection with query timeouts to release resources fast even for unsuccessful queries. - Set a maximum number of connections allowed, with a queue. - Set a limit to the memory used by each query. - Monitor disk and CPU usage.
8	Citizen PII stored unencrypted	Information disclosure	TBD	Mitigated		If the database disk is unencrypted and obtained by a Bad Actor, the data can be read directly.	- Encrypt the database at rest. - Keep the database isolated to the app host - don't serve access to the database over the network
9	No audit trail of account data changes	Repudiation	TBD	Mitigated		Without an audit log, a citizen could deny changes they have made to their account credentials. There is also no way of tracking when and whether a Bad Actor has submitted changes to a citizens account.	- Maintain an audit log of account change events such as registration and verification. - Include timestamps in logs - Monitor logs for abnormalities

Register (E1) (Process)

Description: The register process which takes the citizen's email and password, hashes the password, generates a verification code and creates the account.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
10	Flooding the register process with get requests	Denial of service	TBD	Mitigated		A bad actor could send many requests to the server because it is a process that does not require authentication.	- Rate limiting for requests from the same IP on the reverse proxy. - Add load balancing to reverse proxy to spread request traffic across servers.
11	Registration process replaced by scammer	Spoofing	TBD	Accepted		The official registration page has been replaced by another malicious page users are redirected to from scam emails, or text messages, or links posted online.	- Buy similar domain names with typos.
12	Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		Process logs sensitive data, or stores data in unsafe (e.g., shared) memory	- Use type-safe and memory-safe programming language e.g. Java, Rust - Ensure log entries don't contain sensitive data. - Monitor process usages and in/out for suspicious activity.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
13	Process has been tampered with	Tampering	TBD	Mitigated		The process has been altered by an external malicious actor. This can be done by modifying source code and binaries (static tampering), injecting a different process (DLL injection), or memory corruption that can disrupt execution flow (e.g. corrupting heap metadata like function pointers).	- Monitor and keep a record of process ID to identify process restart. - Add fingerprinting to process, e.g., change time, binary / script hash. - Monitor process in/out and flag unexpected data patterns, malformed payloads unusual destinations or processes that deviate from usual register behaviour.
14	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		Process runs at a higher priority level (e.g., permissions, resources), or accesses to assets outside its permission level.	- Apply principle of least privilege for process and used resources. - Run process on the OS with as few permissions as required, using a dedicated technical account rather than a normal user account with interactive and logon permissions. - Run in container with dropped capabilities.
15	Injection attack - tampering	Tampering	TBD	Mitigated		Malicious data is passed to the process to cause harm to the underlying data, e.g., override data or inject malicious data.	- Decode, normalise, and canonicalise email and password before validation and storage - Sanitise input for script-like requests, e.g., javascript, SQL, etc. - Enforce strict Content Security Policy to prevent harmful scripts to be transmitted (e.g., js, py, sh files). - Add parameterised queries/ prepared statements for all DB access.
16	Injection attack - disclosure	Information disclosure	TBD	Mitigated		Malicious data is passed to the process to gain access to or exfiltrate data stored in database.	- Decode, normalise, and canonicalise email and password before validation and storage - Sanitise input from script-like statements, e.g., javascript, SQL, etc. - Enforce strict CSP policy to prevent harmful scripts to be transmitted (e.g., js, py, sh files).
17	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		Logs may leak sensitive data such as email, password, IP, and session IDs, when tracing user actions.	- Ensure that all sensitive PII data other than user ID are not logged. - Conduct regular log audits.

hashed_password (Data Flow)

Description: The hashed password returned to the register process to be stored with the citizen's account.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

verification_code (Data Flow)

Description: The verification code generated to verify the citizen's email address.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

hashed_given_password (Data Flow)

Description: The hashed password provided to the validation process for comparison against the stored credentials.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

unique_session_id (Data Flow)

Description: A unique session ID generated for the citizen's session after a successful login.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

submit_register_request (Data Flow)

Description: The citizens email and password they want to use to register an account.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
18	Citizen personal data is tampered with in transit	Tampering	TBD	Mitigated		Bad actor changes the citizen's email and/or password so they either do not receive the verification email, or can't successfully log in.	- Use HTTPS (or another encrypted protocol) to send data over the network
19	Citizen information exposed to a bad actor	Information disclosure	TBD	Mitigated		Email address and password is accessed by a bad actor on the network, allowing them to steal the user's credentials.	- Use HTTPS (or another encrypted protocol) to send data over the network - Use verification code to ensure stolen credentials cannot be used to log in.
20	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	

submit_register_response (Data Flow)

Description: Successful/unsuccessful registration response from the server.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
21	Error messages reveal information about system state	Information disclosure	TBD	Mitigated		An error message could contain information such as - server software & version - file system paths - stack traces - sensitive information such as email and password	- Don't have revealing error messages - Use HTTPS (or another encrypted protocol) to send data over the network.
22	Register process response is changed in transit	Tampering	TBD	Mitigated		See #64	

get_login_response (Data Flow)

Description: Response from the login process containing the html, css, js etc of the login page.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
23	Login process response changed in transit	Tampering	TBD	Mitigated		See #64	
24	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	

register_citizen_info (Data Flow)

Description: The citizen's email and hashed password sent to be persisted in the citizen information data store.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
25	Data or query altered in transit	Tampering	TBD	Mitigated		Bad actor alters query in transit (e.g. adding DROP tables to the query) or alters the data in the query to be incorrect (e.g. updating the query to set is_verified = true)	- Use local private network. - Use certificates to authenticate originating machine. - Use prepared statements on the caller side to minimise chances of effects of tampered data. - Use TLS/SSL encryption for all database connections.
26	Data is read in transit	Information disclosure	TBD	Mitigated		A bad actor captures the data in transit	- Use a secure private network - Use TLS/SSL encryption for all database connections.
27	Wire is overloaded by request	Denial of service	TBD	Mitigated		The wire is overloaded by requests in the private network.	- Monitor network activity for suspicious patterns, e.g., corrupted processes - Drop suspicious packets and ban originating IPs (local firewalling)

password (Data Flow)

Description: The citizen's chosen password sent to the hashing process.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
28	Plaintext password handled in application memory	Information disclosure	TBD	Mitigated		The password is passed in plaintext between application processes/components. A component that logs its inputs or dumps memory on error could leak the password before it is hashed.	- Minimise the number of components that touch a plaintext password by hashing as close to the entry point as possible, - Don't log request bodies or wipe passwords before logging. - Apply the principle of least privilege to each component.

verification_email (Data Flow)

Description: The verification email is routed to the citizens email mailbox by the MTA

Number	Title	Type	Severity	Status	Score	Description	Mitigations
29	Verification link intercepted in transit via SMTP	Information disclosure	TBD	Open		If the email is not encrypted end to end, a Bad Actor could observe an unencrypted verification email.	- Use end-to-end encryption - Invalidate the link once used by the citizen

Email Service Provider (D10) (Process)

Description: The email service provider that sends the verification email on behalf of the application.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
30	Verification email spoofed as coming from SecureReports	Spoofing	TBD	Mitigated		The absence of SPF/DKIM/DMARC configuration allows Bad Actors to forge a verification email as coming from SecureReports.	- Configure SPF, DKIM, DMARC on the domain used to send verification emails - Monitor DMARC reports for unauthorised sending attempts
31	Email vendor becomes compromised through tampering	Tampering	TBD	Mitigated		The email service and is assumed third-party infrastructure. If the vendor is compromised (e.g. its API keys are stolen), an attacker can send mail as SecureReports, or read mail in transit.	- Apply the principle of least privilege when storing API keys - Rotate keys regularly - Audit the third-party software's security - Store keys in a secret vault
32	Phishing SecureReports verify email clone	Spoofing	TBD	Accepted		A Bad Actor impersonates the legitimate SecureReports verification email with a link that directs citizens to a illegitimate site that harvests their email, password, and/or other personal data.	- Purchase all look-alike domains.
33	Verification token is retained in provider logs	Information disclosure	TBD	Mitigated		If the email provider logs message content/ metadata, it can become exposed later if the email provider is compromised.	- Configure the provider's log retention period to a minimum - Conduct regular security audits of the third-party email provider - Invalidate the link once used by the citizen

Hashing (D7) (Process)

Description: The process/algorithm that hashes a user's password

Number	Title	Type	Severity	Status	Score	Description	Mitigations
34	Library replaced with a fake/ malicious library	Spoofing	TBD	Mitigated		See #48	
35	Process has been tampered with	Tampering	TBD	Mitigated		See #13	

Number	Title	Type	Severity	Status	Score	Description	Mitigations
36	Injection attack - tampering	Tampering	TBD	Mitigated		See #15	
37	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		See #14	
38	Weak hashing algorithm allows reversal or collision	Information disclosure	TBD	Mitigated		The hashing algorithm used to store passwords may not be a modern password-hashing function, or may have known weaknesses (e.g., collisions, fast computation enabling brute force, no built-in salting), allowing a bad actor with access to a hashed value to recover the original password or forge a matching hash.	- Use a purpose-built password hashing algorithm (e.g., bcrypt, scrypt, Argon2) rather than a general-purpose hash function (e.g., MD5, SHA-1, unsalted SHA-256). - Apply a unique salt per password to prevent rainbow table attacks. - Use an appropriate work factor/cost parameter and increase it over time as hardware improves. - Avoid algorithms with known collision vulnerabilities. - Add constant time hash comparison to avoid revealing information about the hashed content via timing differences

Verify Email (E2) (Process)

Description: The action to verify a user account using the code sent by email

Number	Title	Type	Severity	Status	Score	Description	Mitigations
39	Flooding the verify email process with requests	Denial of service	TBD	Mitigated		See #10	
40	Verification process replaced by scammer	Spoofing	TBD	Accepted		See #11	
41	Process has been tampered with	Tampering	TBD	Mitigated		See #13	
42	Injection attack - tampering	Tampering	TBD	Mitigated		See #15	
43	Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		See #12	
44	Injection attack - disclosure	Information disclosure	TBD	Mitigated		See #16	
45	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		See #17	
46	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		See #14	
47	Bad actor verifies email using stolen verification link	Spoofing	TBD	Open		Bad actors finds a verification link (through measures like phishing and sniffing) and uses it to verify an account made with an email they do not own.	

Random Number Generator (D8) (Process)

Description: The dependency process that generates randoms to be used for generating session ids, verification code, and seeds.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
48	Library replaced with a fake/ malicious library	Spoofing	TBD	Mitigated		Legitimate library replaced with a fake library by an actor, so consuming applications unknowingly pull numbers from an attacker controlled source.	- Code signing - require the library to be crypto-graphically signed by a trusted authority, and verify this signature before load. - Add fingerprinting to process, e.g., change time, binary / script hash, and verify at load time or periodically at runtime. - Use fixed absolute library paths for loading. - Use an allow list to declare exactly which library version and path are permitted - Monitor unexpected restarts or new processes claiming to be the library.
49	Entropy/seed leakage	Information disclosure	TBD	Mitigated		Internal state or seed value of the random number generator is exposed, allowing a bad actor to reconstruct past or predict future random numbers.	- Choose a well audited library (and library version) with no reported vulnerabilities. - Monitor the 'health' of the entropy source
50	Generation algorithm makes random number guessable	Information disclosure	TBD	Mitigated		Random number generator uses an algorithm that is not cryptographically secure, meaning even without the internal state being known, a past and future output can be guessed using brute force.	- Prefer libraries that use algorithms specified in NIST SP 800-90A/B/C or equivalent, which have been formally analyzed for prediction resistance. - Ensure that library uses an algorithm such that compromise of current state does not allow reconstruction of past outputs or prediction of future outputs.
51	Process has been tampered with	Tampering	TBD	Mitigated		See #13	
52	Injection attack - tampering	Tampering	TBD	Mitigated		See #15	- Choose a well audited library (and library version) with no reported vulnerabilities.

Login (E3) (Process)

Description: The action to log a citizen into the SecureReports app

Number	Title	Type	Severity	Status	Score	Description	Mitigations
53	Flooding the process with requests	Denial of service	TBD	Mitigated		See #10	
54	Process replaced by scammer	Spoofing	TBD	Accepted		See #11	Provide remediation for this threat or a reason if status is N/ A
55	Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		See #12	
56	Process has been tampered with	Tampering	TBD	Mitigated		See #13	

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		See #14	
58	Injection attack - tampering of underlying data	Tampering	TBD	Mitigated		See #15	
59	Injection attack - disclosure	Information disclosure	TBD	Mitigated		See #16	
60	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		See #17	
61	Spoofed login session	Spoofing	TBD	Mitigated		Stolen cookies remain valid indefinitely, letting an attacker impersonate the citizen long after the cookie was stolen	- Have an idle-timeout and absolute-timeout for all session IDs. - Invalidate a session ID once logout occurs.
62	Non secure session cookie sent back to the Citizen	Spoofing	TBD	Mitigated		If the response from the login process fails to set the HttpOnly flag on the session cookie, a malicious script running on the front end could read/write to the cookie and allow a Bad Actor to impersonate the citizen.	- Use the HttpOnly + Secure flag on cookies to prevent client side javascript code from reading to the cookie. This also prevents writing in most browsers.
63	Account enumeration via error messages	Information disclosure	TBD	Mitigated		The application returns different responses or takes a different amount of time to respond depending on whether an email address is registered, has the wrong password, or hasn't been verified yet. This information can be used to determine which email addresses have accounts on the system.	- Return the same comprehensive error message for all failure cases. - Add timing delays so that timing differences are not observable.

get_register_response (Data Flow)

Description: The register page html, css, js etc.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
64	Register process response is changed in transit	Tampering	TBD	Mitigated		Response html/javascript is tampered with by a bad actor e.g., injecting a malicious script, altering the form's action URL to point to an attacker-controlled endpoint, or adding a hidden field.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit

given_password (Data Flow)

Description: The password given at registration sent to the hashing process to be hashed for comparison.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
65	Plaintext password handled in application memory	Information disclosure	TBD	Mitigated		The password is passed in plaintext between application processes/components. A component that logs its inputs or dumps memory on error could leak the password before it is hashed.	- Minimise the number of components that touch a plaintext password by hashing as close to the entry point as possible, - Don't log request bodies or wipe passwords before logging. - Apply the principle of least privilege to each component.

verification_info (Data Flow)

Description: Verify email process receives the verification code from the Citizen information store, and sends the is_verified status of the email once verified.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Data or query altered in transit	Tampering	TBD	Mitigated		See #25	
67	Data is read in transit	Information disclosure	TBD	Mitigated		See #26	
68	Wire is overloaded by request	Denial of service	TBD	Mitigated		See #27	

verification_email (Data Flow)

Description: The verification email is passed to the receiving message transfer agent to route to the recipient.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Receiving MTA (Process)

Description: This process represents the mail transfer agent who routes the email from the server email service provider to the recipients email service provider using SMTP

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	MTA becomes compromised	Spoofing	TBD	Mitigated		The MTA is assumed third-party infrastructure. If the vendor is compromised (e.g. its API keys are stolen), an attacker can send mail as SecureReports.	- Apply the principle of least privilege when storing API keys - Rotate keys regularly - Audit the third-party software's security

Citizen Mailbox (Store)

Description: The citizen's email mailbox where the verification email is delivered and read.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
70	Verification link read from a shared mailbox	Information disclosure	TBD	Mitigated		The verification email may be read by anyone with access to the citizen's mailbox. A Bad Actor may click on the link to also verify the citizen's account and gain information though the verification flow.	- Invalidate verification links once used by the citizen

user_info (Data Flow)

Description: Given a user's email, the login process requests the hashed password and if they are verified (is_verified) from the datastore

Number	Title	Type	Severity	Status	Score	Description	Mitigations
71	Data or query altered in transit	Tampering	TBD	Mitigated		See #25	
72	Data is read in transit	Information disclosure	TBD	Mitigated		See #26	
73	Wire overloaded by requests	Denial of service	TBD	Mitigated		See #27	

get_register_request (Data Flow)

Description: A request for the register process to show the citizen the register page

Number	Title	Type	Severity	Status	Score	Description	Mitigations
74	Flooding the network with traffic	Denial of service	TBD	Mitigated		A bad actor floods the request channel with high-volume traffic, exhausting network/connection capacity and blocking legitimate requests from reaching the server.	- Rate limiting for requests from the same IP.
75	Citizen register request is changed in transit	Tampering	TBD	Mitigated		Request URL query parameters or body is tampered with by a bad actor e.g., injecting a malicious redirect or return parameter.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit

get_login_request (Data Flow)

Description: Request to the login process to show the login page

Number	Title	Type	Severity	Status	Score	Description	Mitigations
76	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	
77	Citizen login request is changed in transit	Tampering	TBD	Mitigated		See #75	

submit_login_request (Data Flow)

Description: Citizen's email and password for their account

Number	Title	Type	Severity	Status	Score	Description	Mitigations
78	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	
79	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		See #19	
80	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the login credentials that the citizen supplied and log them into an account that is not theirs. Subsequent actions such as report making will then be performed on the attackers account.	- Use HTTPS (or another encrypted protocol) to send data over the network

verify_email_request (Data Flow)

Description: Verification code

Number	Title	Type	Severity	Status	Score	Description	Mitigations
81	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	
82	Citizen verify request is changed in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the verification code in the request to be invalid and prevent the Citizen from verifying their account.	- Use HTTPS (or another encrypted protocol) to send data over the network

redirect_to_login_response (Data Flow)

Description: Successful/unsuccessful verification response from the server plus redirect to login on success.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
83	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	
84	Verify Email process response is changed during transit	Tampering	TBD	Mitigated		See #64	

submit_login_response/session_cookie (Data Flow)

Description: Successful/unsuccessful login response from the register process and necessary redirect.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
85	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #74	

Number	Title	Type	Severity	Status	Score	Description	Mitigations
86	Citizen session cookie exposed in transit	Information disclosure	TBD	Mitigated		A Bad Actor could observe the session cookie assigned to the Citizen on log-in and use the session-cookie to perform actions on behalf of the Citizen.	- Use HTTPS (or another encrypted protocol) to send data over the network. Use the Secure flag on cookie to prevent it from being sent over unencrypted HTTP - Rotate session Cookies often
87	Login process response altered in transit	Tampering	TBD	Mitigated		If a Bad Actor observed and altered the response from the login process to a different session cookie. The Citizen may be logged into an account owned by the Bad Actor. The Bad Actor could also alter the response to redirect the Citizen to a malicious website or to a spoofed version of SecureReports.	- Use HTTPS (or another encrypted protocol) to send data over the network

verification_link (Data Flow)

Description: The verification link sent by the register process to the email service provider, to be emailed to the citizen.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
88	Verification link altered in transit	Tampering	TBD	Mitigated		A Bad Actor may observe and modify the verification link in transit, swapping the token to prevent a user from verifying their account, or altering the url so the citizen will be brought to a phishing page.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit
89	Registration flood exhausts the email API	Denial of service	TBD	Mitigated		Mass automated registrations can cause the Server's API limit to be reached or overload the email service provider	- Rate limit the registration request before the call to the ESP is performed. - Monitor API usage limits - Log traffic and audit logs regularly to detect anomalies.

email_address (Data Flow)

Description: The citizen's email address sent to the email service provider so the verification email can be sent.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
90	Citizens verification email is altered in transit	Tampering	TBD	Mitigated		Verification email is altered in transit by a Bad Actor and replaced by an email they own, so the citizen's verification link is sent to them instead.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit
91	Registration flood exhausts the Email API	Denial of service	TBD	Mitigated		See #89	

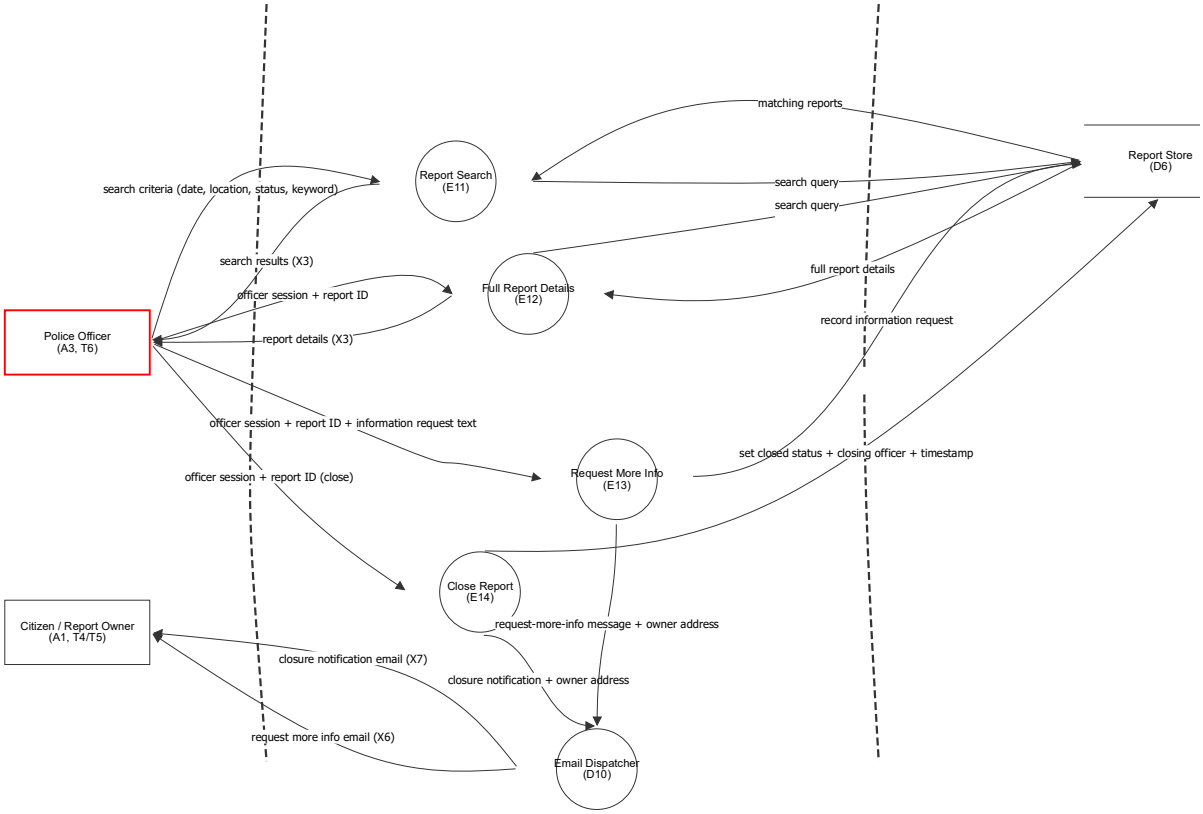
verification_email (Data Flow)

Description: Verification email containing the verification code as read from the citizens mailbox

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
92	Verification link visible on screen	Information disclosure	TBD	Mitigated		If the email displays the verification link on screen, it could be seen by a Bad Actor who then uses it to verify and exploit more information about the citizen's account through the verification flow.	- Obscure the verification link in the email as a hyperlinked button or styled link text rather than displaying the raw URL.

Reports - Officer Access (E11-E14)



Reports - Officer Access (E11-E14)

Police Officer (A3, T6) (Actor)

Description: A3 Police officer. T6 logged-in police officer: searches and views all reports, emails owners requesting more information and closes reports. Assumed unable to create or edit reports.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
158	Officer account compromised	Spoofing	High	Open		An attacker holding officer credentials, or an intercepted officer session, reads every report in the system and can close any of them.	Serve officer routes over HTTPS with HSTS, apply a short idle timeout and alert on unusual sign in locations.
159	Privileged officer action not recorded	Repudiation	High	Mitigated		There is no record of which officer performed a privileged action like closures or requesting further information	Write an audit record of officer ID, action, related text, timestamp, and protect the logs (X9) from the T6 role.

Citizen / Report Owner (A1, T4/T5) (Actor)

Description: A1 Citizen. T4 logged-in citizen, T5 owner of the reports they created. Registers, submits reports with images, views and edits their own open reports and generates share links.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
160	Report owner's mailbox controlled by an attacker	Spoofing	Medium	Mitigated		An attacker who controls the report owner's mailbox receives the information request (X6) and the closure notification (X7)	Re-verify the address whenever it is changed, and keep incident detail out of email by linking back to the authenticated page.

Report Search (E11) (Process)

Description: E11 Report search page. The form where an officer searches reports by date, location, status or keyword. Returns exit point X3.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
161	Search injection	Tampering	High	Mitigated		Keyword and filter values are attacker influenced input. If they are concatenated into the query rather than bound, an attacker can read or alter data in D6.	Parameterised queries and treat every search term as data, never as query syntax.
162	Search accessible without the officer role	Elevation of privilege	High	Mitigated		A logged-in citizen (T4) reaching the search route would obtain reports belonging to everyone.	Enforce the T6 role server-side on the route itself, not only by hiding the navigation link.
163	Unbounded search scans the whole store	Denial of service	Medium	Mitigated		Leading wildcard or empty-criteria searches force a full scan of every report in D6, denying the service to other users.	Paginate results, apply query timeouts and index the searchable columns.

Full Report Details (E12) (Process)

Description: E12 Officer report details page. The page where an officer views the full details of any report. Returns exit point X3.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
164	Bulk harvesting of citizen reports	Information disclosure	High	Mitigated		An officer account enumerates and exports every report, including incident locations and images	rate limit searches, log every access with officer ID, and alert on abnormal viewing volume.
165	Role check missing on the detail page	Elevation of privilege	High	Mitigated		The detail route is reachable by a non-officer session if authorisation is only applied at menu rendering.	Verify the T6 role server-side on every request to the officer detail route.

Request More Info (E13) (Process)

Description: E13 Request more info form. The form where an officer provides the extra details required, which the system emails to the report owner. Produces exit point X6.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
166	Email header injection	Tampering	High	Mitigated		Injecting extra headers, letting the message be redirected or additional recipients added.	Include input validataion and construct headers through the mail library rather than by string building.
167	Information request sent to the wrong owner	Information disclosure	High	Mitigated		The recipient address is taken from the request instead of the stored report, so report detail reaches an attacker chosen mailbox.	Resolve the recipient from the report's stored owner in D6, never from client supplied input.
168	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		Mail delivery to the citizen's external provider is outside the system's control so any incident detail ends up in an external mailbox in cleartext.	Keep incident detail out of the message body, send a short notice identifying the report by reference with a link back to the authenticated page

Close Report (E14) (Process)

Description: E14 Close report action. The action which closes a report and notifies its owner. Produces exit point X7.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
169	Unauthorised closure of a report	Elevation of privilege	High	Mitigated		A crafted request from a citizen session closes a report, which would also stop the owner editing it (see E7).	- Enforce the T6 role server-side on the close action and reject the closed field on any citizen facing endpoint. - CSRF protection.
170	Privileged officer action not recorded	Repudiation	Medium	Mitigated		E14 is where the closure is written and must record who set it. See the identically titled threat on Police Officer (A3, T6)	See the identically titled threat on Police Officer (A3, T6).
171	Cross site request forgery closes a report	Spoofing	High	Mitigated		A closure is irreversible for the owner, who can no longer edit the report afterwards.	- Require a CSRF token on the close action. - Use SameSite cookies and check the Origin/Referer header

Email Dispatcher (D10) (Process)

Description: D10 Email dispatcher. Sends the verification, information request and closure emails to the citizen's external email provider.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
172	Mail credentials disclosed	Information disclosure	High	Mitigated		Mail service credentials stored in the application configuration are read by an attacker, who can then send mail as the service.	- Keep configuration outside the web root, readable only by the T7 account, and rotate credentials. - Use secure secrets vault and apply the principle of least privilege.
173	Dispatcher abused to send arbitrary mail	Spoofing	High	Mitigated		An attacker who can reach the dispatcher sends convincing mail that appears to come from the service, enabling phishing against citizens.	- Authenticate which accounts have access to the mail service,
174	Email queue flooded	Denial of service	Medium	Mitigated		Repeated information requests or closures generate enough mail to exhaust the sending quota, delaying verification email (X5) and blocking registration.	- Rate limit outbound mail per officer and per report. - Monitor queue size for abnormalities

Report Store (D6) (Store)

Description: D6 Database. Asset: report content - date, time, location, description, created time, closed status and owner / created_by. Reachable only from the application (T7 -> T8).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
175	Unauthorised report disclosure	Information disclosure	High	Mitigated		Direct database access could expose sensitive report information.	Restrict database access, use least privilege and protect data at rest.
176	Direct report tampering	Tampering	High	Mitigated		Database access could bypass application authorisation and modify reports.	Restrict direct database access and audit administrative changes.

closure notification email (X7) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
177	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		The closure notification leaves the system on this flow and may quote the incident. See the identically titled threat on Request More Info (E13).	See the identically titled threat on Request More Info (E13).

search criteria (date, location, status, keyword) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
178	Search injection	Tampering	High	Mitigated		This flow carries the attacker-influenced search terms into E11. See the identically titled threat on Report Search (E11).	See the identically titled threat on Report Search (E11).

officer session + report ID + information request text (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
179	Information request text observed in transit	Information disclosure	Medium	Mitigated		The text may quote incident detail from the report.	Use HTTPS protocol only with HSTS .

officer session + report ID (close) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
180	Report identifier substituted to close an unrelated report	Tampering	Medium	Mitigated		The report ID is client supplied and could be changed to close a report the officer did not intend.	Record the closing officer and timestamp for audit.

officer session + report ID (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
181	Role check missing on the detail page	Tampering	High	Mitigated		This flow carries the session and the client supplied report ID that E12 must authorise on every request. See the identically titled threat on Full Report Details (E12).	See the identically titled threat on Full Report Details (E12).

report details (X3) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
182	Bulk harvesting of citizen reports	Information disclosure	High	Mitigated		Every detail view leaves the system on this flow, so repeated views are how the harvesting actually happens. See the identically titled threat on Full Report Details (E12).	See the identically titled threat on Full Report Details (E12).

search results (X3) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
183	Bulk harvesting of citizen reports	Information disclosure	Medium	Mitigated		Repeated wide searches harvest the database through this flow. See the identically titled threat on Full Report Details (E12).	See the identically titled threat on Full Report Details (E12).

request more info email (X6) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
184	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		The information request email leaves the system on this flow. See the identically titled threat on Request More Info (E13).	See the identically titled threat on Request More Info (E13).

closure notification + owner address (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
185	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		This flow hands the closure notification and the owner's address to D10 for delivery outside the system. See the identically titled threat on Request More Info (E13).	See the identically titled threat on Request More Info (E13).

request-more-info message + owner address (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
186	Email header injection	Tampering	High	Mitigated		This flow hands the officer supplied text and recipient address to D10 where injected header lines would take effect. See the identically titled threat on Request More Info (E13).	See the identically titled threat on Request More Info (E13).

set closed status + closing officer + timestamp (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
187	Privileged officer action not recorded	Tampering	Medium	Mitigated		This flow is the closure write that must carry the closing officer and timestamp in the same statement. See the identically titled threat on Police Officer (A3, T6).	See the identically titled threat on Police Officer (A3, T6).

record information request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
188	Privileged officer action not recorded	Tampering	Medium	Mitigated		This flow is the audit write for an information request by a Police offer. See the identically titled threat on Police Officer (A3, T6).	See the identically titled threat on Police Officer (A3, T6).

full report details (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
189	More columns returned than the page renders	Information disclosure	Medium	Mitigated		The detail read pulls the owner's account identifiers alongside the incident record. See the identically titled threat on matching reports.	See the identically titled threat on matching reports.

search query (Data Flow)

Description: Search query for full report details

Number	Title	Type	Severity	Status	Score	Description	Mitigations
190	Search injection	Tampering	High	Mitigated		The identifier E12 looks up is client supplied and reaches D6 on this flow. See the identically titled threat on Report Search (E11).	See the identically titled threat on Report Search (E11).

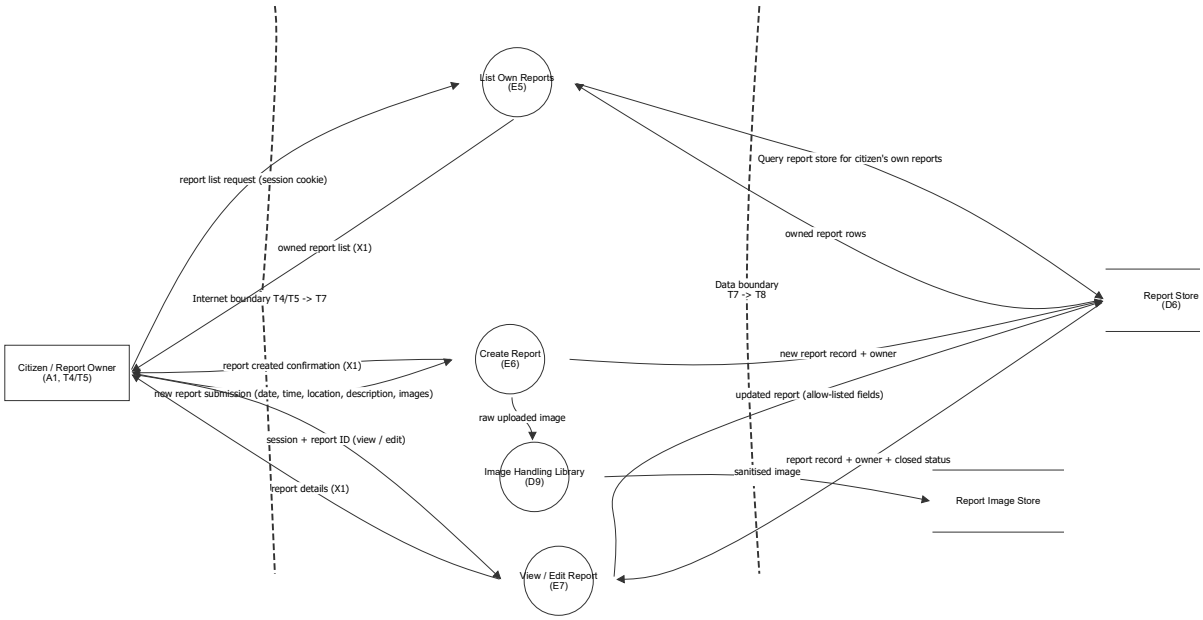
matching reports (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
191	More columns returned than the page renders	Information disclosure	Medium	Mitigated		Returning every column of every match exposes citizen account identifiers alongside the incident record.	Return only the columns the page renders, no citizen personal data.

search query (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
192	Unbounded search scans the whole store	Denial of service	Medium	Mitigated		This flow is the query that reaches D6. See the identically titled threat on Report Search (E11) for the full description.	See the identically titled threat on Report Search (E11).

Reports - Owner Access (E5-E7)



Reports - Owner Access (E5-E7)

Citizen / Report Owner (A1, T4/T5) (Actor)

Description: A1 Citizen. T4 logged-in citizen, T5 owner of the reports they created. Registers, submits reports with images, views and edits their own open reports and generates share links.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
93	Session cookie theft	Spoofing	High	Mitigated		A stolen session identifier lets an attacker act as the report owner (T5) without the password.	Rotate the session ID on login, apply an idle timeout and offer logout (E4).
94	Citizen denies submitting a report	Repudiation	Medium	Mitigated		The owner of a report later denies having created or edited it.	Record owner ID and timestamp on every create and edit, and retain application logs (X9) with restricted access.

List Own Reports (E5) (Process)

Description: E5 Report list page. Lists reports owned by the citizen and their status. Returns exit point X1.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
95	Report list not scoped to owner	Information disclosure	High	Mitigated		A citizen could receive other reports if the query is not constrained to the authenticated owner.	- Derive ownership from the session and enforce in the database query.

Create Report (E6) (Process)

Description: E6 Create report page. Citizen enters the date, time, location, description and images of a new report.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
96	Malicious image upload	Elevation of privilege	High	Mitigated		Anything the citizen uploads is handed to the image handling library, meaning a malicious file can exploit the decoder before any content check.	- Verify the bytes against permitted formats before decoding, reject on mismatch, and process images in a low-privilege sandbox. - Decode in a sandboxed/containerised environment
97	Image metadata disclosure	Information disclosure	High	Mitigated		If stripping fails or is skipped for a format, the image written to the store still carry EXIF metadata such as GPS coordinates.	- Strip metadata by re-encoding rather than copying, and assert on the artefact that no EXIF or GPS block remains. - Never re-add metadata
98	Oversized image denial of service	Denial of service	High	Mitigated		Large or resource-intensive images can consume storage and processing resources.	- Enforce upload size, dimension and processing limits. - Use per-upload timeouts
99	Owner field taken from the request	Elevation of privilege	High	Mitigated		If created_by is accepted from the payload, a citizen can file a report attributed to someone else.	- Set the owner from the authenticated session only and reject the field if it appears in the request. - Use an allow-list of writable fields

View / Edit Report (E7) (Process)

Description: E7 View and edit report page. The report owner (T5) views and edits their own report. Returns exit point X1.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
100	Insecure direct object reference	Elevation of privilege	High	Mitigated		A user could change the report ID to access or edit another citizen's report if the system does not check who owns it.	- Derive the owner from the authenticated session and perform a server-side ownership check on the identifier for every report request.
101	Editing closed reports	Tampering	High	Mitigated		A crafted request could bypass a hidden edit button if the server does not enforce report state, so a closed report can still be modified.	- Always read the closed status with the record and reject updates server-side when the report is closed.
102	Mass assignment of closed status	Elevation of privilege	High	Mitigated		A citizen may attempt to manipulate protected fields such as closed status.	- Allow-list editable fields and never accept closed status from the citizen.
103	Stored cross-site scripting in report description - Tampering	Tampering	High	Mitigated		A user could enter malicious script in a report description that runs when the report is viewed, and tampers with page content and behaviour.	- Safely encode the report description when displaying it so it is treated as text rather than code. - Use a strict CSP to prevent interaction with other domains - Use HttpOnly cookies so that a XSS attack cannot tamper with session ids and other important data.
104	Stored cross-site scripting in report description - Information Disclosure	Information disclosure	High	Mitigated		A user could enter malicious script in a report description that runs when the report is viewed, and tampers with page content and behaviour.	- Safely encode the report description when displaying it so it is treated as text rather than code. - Use a strict CSP to prevent interaction with other domains - Use HttpOnly cookies so that a XSS attack cannot tamper with session ids and other important data.

Image Handling Library (D9) (Process)

Description: D9 Image handling library. Processes uploaded report images and writes them to storage.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
105	Vulnerable image parser	Elevation of privilege	High	Mitigated		A crafted image exploits a decoder flaw in D9 and executes code as the T7 application identity.	- Restrict decoding to an allow-list of formats, and process images in a low-privilege sandboxed worker. - Regularly audit image library security and bump versions when patches are released
106	Decompression bomb exhausts memory	Denial of service	High	Mitigated		A small file that expands to an enormous bitmap consumes all memory and CPU during processing.	Enforce maximum pixel dimensions and total pixel count before decoding, plus per-image processing timeouts and memory caps.
107	Image path derived from client input	Tampering	High	Mitigated		A user could manipulate the image filename to access or store files outside the intended image storage location.	- Generate image filenames on the server and only allow files to be stored and retrieved from the designated image directory. - Use a secure random process for generating filenames. - Decode, normalise, and canonicalise if user input is necessary.

Report Store (D6) (Store)

Description: D6 Database. Asset: report content - date, time, location, description, created time, closed status and owner / created_by. Reachable only from the application (T7 -> T8).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
108	Unauthorised report disclosure	Information disclosure	High	Mitigated		Direct database access could expose sensitive report information.	- Restrict database access, use least privilege and protect data at rest.
109	Direct report tampering	Tampering	High	Mitigated		Database access could bypass application authorisation and modify reports.	- Restrict direct database access and audit administrative changes.

Report Image Store (Store)

Description: Asset: report images. Uploaded photographs related to an incident, written by D9 and read back by E10. Held outside the web root.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
110	Unauthorised image retrieval	Information disclosure	High	Mitigated		Image files can be read straight from storage, bypassing the report-level authorisation performed by E10.	- Keep the image store outside the web root so files are never served directly; only E10 may read it, using the T7 application identity.

report list request (session cookie) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
111	Session cookie intercepted in transit	Information disclosure	High	Mitigated		The session identifier authenticates every subsequent request, observed on the network it allows full impersonation of the report owner (T5).	- Use HTTPs only and a short idle timeout.
112	Request flooding on the report list	Denial of service	Medium	Mitigated		Repeated list requests force repeated database queries and deny the service to other citizens.	- Per-session and per-IP rate limiting in the application.

owned report list (X1) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
113	Report data cached by the browser or an intermediary	Information disclosure	High	Mitigated		A cached listing or report page can be recovered from a shared machine after logout, or served to another user by a shared cache.	Disable caching for report routes at the assumed reverse proxy (D3).
114	Error responses reveal application state	Information disclosure	Medium	Mitigated		Stack traces or framework error pages disclose D5 versions and file system paths.	Return generic error pages and log the detail server-side only (X9).

new report submission (date, time, location, description, images) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
115	Report content and incident location observed in transit	Information disclosure	High	Mitigated		The submission carries the incident location and description, which are sensitive to the reporting citizen.	HTTPS only with HSTS (D2/ D3, see model assumptions).
116	Report altered in transit	Tampering	High	Mitigated		An attacker on the path modifies the incident detail or substitutes the uploaded images before they reach E6.	Use HTTPS/TLS to protect the data while it is being transmitted.
117	Oversized multipart upload exhausts capacity	Denial of service	Medium	Mitigated		A very large upload ties up connections and disk before E6 can reject it.	Enforce a request body size limit and an upload timeout.

report created confirmation (X1) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
118	Confirmation discloses internal identifiers	Information disclosure	Medium	Mitigated		Returning database keys or storage paths in the report creation response tells an attacker how to address other records.	Return only the fields the citizen needs with no internal identifiers and no stack traces, use generic error messages.

session + report ID (view / edit) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
119	Insecure direct object reference	Tampering	High	Mitigated		This flow carries the client-supplied report ID that can be altered. See the identically titled threat on View / Edit Report (E7) for the full description.	See the identically titled threat on View / Edit Report (E7).
120	Edited report content observed in transit	Information disclosure	High	Mitigated		Both the retrieved report and the edited version cross the internet boundary in this exchange.	HTTPS only with HSTS (see model assumptions).

report details (X1) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
121	Report data cached by the browser or an intermediary	Information disclosure	High	Mitigated		The report detail response is authenticated content crossing the internet boundary. See the identically titled threat on owned report list (X1) for the full description.	See the identically titled threat on owned report list (X1).

report record + owner + closed status (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
122	Editing closed reports	Tampering	Medium	Mitigated		This flow is where the closed status must be carried into the edit decision. See the identically titled threat on View / Edit Report (E7) for the full description.	See the identically titled threat on View / Edit Report (E7).

sanitised image (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
123	Image metadata disclosure	Information disclosure	High	Mitigated		The image on this flow is only sanitised if stripping actually succeeded for its format. See the identically titled threat on Create Report (E6) for the full description.	See the identically titled threat on Create Report (E6).
124	Image path derived from client input	Tampering	High	Mitigated		The filename carried on this flow is used to build the write path. See the identically titled threat on Image Handling Library (D9) for the full description.	See the identically titled threat on Image Handling Library (D9).

raw uploaded image (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
125	Malicious image upload	Tampering	High	Mitigated		This flow is the unvalidated file as it reaches D9. See the identically titled threat on Create Report (E6) for the full description.	See the identically titled threat on Create Report (E6).

updated report (allow-listed fields) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
126	SQL injection on the report write path	Tampering	High	Mitigated		This flow carries the update values. See the identically titled threat on new report record + owner for the full description.	See the identically titled threat on new report record + owner.

owned report rows (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
127	Over-broad result set returned to the application	Information disclosure	Medium	Mitigated		Selecting whole rows pulls back columns the list page never renders, widening what a later flaw could leak.	Select only the columns the listing needs.

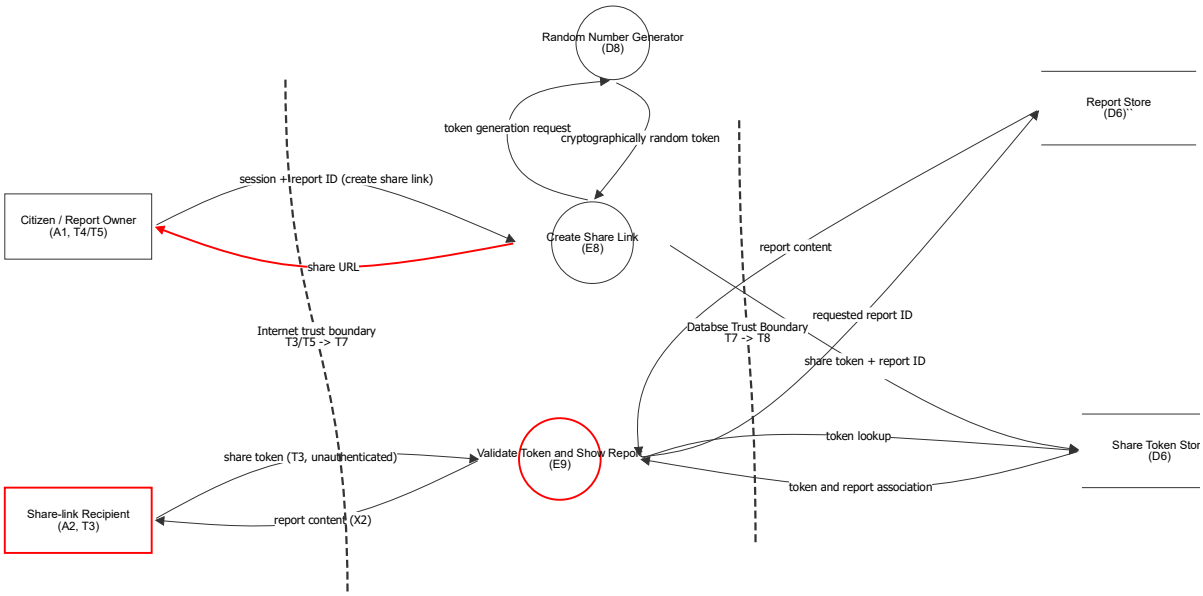
Query report store for citizen's own reports (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
128	Owner predicate omitted or injected	Tampering	High	Mitigated		If the owner constraint is built by string concatenation it can be removed or subverted, returning every citizen's reports.	Parameterised queries with the owner value bound from the session, and never accept the owner from the request.

new report record + owner (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
129	SQL injection on the report write path	Tampering	High	Mitigated		Unparameterised report fields on insert (E6) or update (E7) let an attacker alter or destroy data in D6, including rows beyond their own report.	Use parameterised queries or ORM binding for every insert and update, and give the T7 account only the privileges it needs on D6.

Reports - Share Links (E8-E9)



Reports - Share Links (E8-E9)

Citizen / Report Owner (A1, T4/T5) (Actor)

Description: A1 Citizen. T4 logged-in citizen, T5 owner of the reports they created. Registers, submits reports with images, views and edits their own open reports and generates share links.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
130	Session cookie theft	Spoofing	High	Mitigated		A stolen session lets an attacker create share links for the victim's reports.	HTTPS only, session rotation on login and an idle timeout.

Share-link Recipient (A2, T3) (Actor)

Description: A2 Share-link recipient. T3 unauthenticated user holding a valid share token received outside the system. Never authenticates and is unknown to the system.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
131	Share-link recipient is not authenticated	Spoofing	High	Open		T3 has no identity in the system an attacker holding the URL is indistinguishable from the person the owner meant to share the report with.	- Expose nothing through the share view that assumes the viewer is the intended recipient.

Random Number Generator (D8) (Process)

Description: D8 Random number generator. Produces the share tokens (and the E2 verification codes).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
132	Predictable token	Spoofing	High	Mitigated		A seeded or non-cryptographic generator produces guessable tokens, letting an attacker forge a working share link and read a report without ever being given the link.	Generate share tokens using a cryptographically secure random source, don't use a seeded random number generator

Create Share Link (E8) (Process)

Description: E8 Share link creation. The action which creates the secret link for a report owned by the citizen.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
133	Predictable token	Spoofing	High	Mitigated		E8 gets the token from D8, so it inherits whatever entropy D8 provides. See the identically titled threat on Random Number Generator (D8).	See the identically titled threat on Random Number Generator (D8).
134	Creating a link for someone else's report	Elevation of privilege	High	Mitigated		A citizen submits another citizen's report ID to E8 and obtains a working share link for a report they do not own.	Server side ownership check before creating the token.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
135	Share tokens stored in usable form	Information disclosure	High	Mitigated		See the identically titled threat on Share Token Store (D6).	See the identically titled threat on Share Token Store (D6).
136	Cross-site request forgery creates a share link	Spoofing	High	Mitigated		A third party page causes a logged in owner's browser to create a share link, which the attacker then reads from a page or email.	- Require an CSRF token on the share link action.

Validate Token and Show Report (E9) (Process)

Description: E9 Share link page. The public address that receives a secret token and, if the token is valid, shows the matching report. Returns exit point X2.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
137	Token enumeration	Spoofing	High	Mitigated		An attacker submits large numbers of candidate tokens hoping to hit a valid one. They could also gain information if there is a pattern in the timing in token lookup.	- Use generic responses for invalid tokens - Rate limiting and alerting on sustained invalid token rates. - Constant-time token lookup
138	Closed report remains accessible	Information disclosure	High	Mitigated		A share link created while the report was open keeps returning content after closure if report state is not re-checked during share-link authorisation.	Re-check the report state at read time and refuse the share view for closed reports.
139	Share token remains valid indefinitely once leaked	Elevation of privilege	High	Open		The specified system provides no expiry or revocation for share tokens, so a token that leaks grants access to the report permanently.	- Keep the token out of search indexes, caches and access logs, and rate-limit validation

Report Store (D6)`` (Store)

Description: D6 Database. Asset: report content - date, time, location, description, created time, closed status and owner / created_by. Reachable only from the application (T7 -> T8).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
140	Unauthorised report disclosure	Information disclosure	High	Mitigated		Direct database access could expose sensitive report information.	Restrict database access, use least privilege and protect data at rest.
141	Direct report tampering	Tampering	High	Mitigated		Database access could bypass application authorisation and modify reports.	Restrict direct database access and audit administrative changes.

Share Token Store (D6) (Store)

Description: D6 Database. Asset: share tokens - secret values granting read access to one report without logging in, with their report association, expiry and revocation state.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
142	Share tokens stored in usable form	Information disclosure	High	Mitigated		Anyone who reads the share token store obtains all working share links.	Write only a cryptographic hash of the token, plaintext should exist solely in the URL returned to the owner.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
143	Share token remains valid indefinitely once leaked	Information disclosure	High	Mitigated		See the identically titled threat on Validate Token and Show Report (E9) for the full description.	See the identically titled threat on Validate Token and Show Report (E9).

cryptographically random token (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
144	Predictable token	Information disclosure	High	Mitigated		This flow carries the generated token, so its value is only as unguessable as the source that produced it. See the threat on Random Number Generator (D8) for the full description.	See the identically titled threat on Random Number Generator (D8).

session + report ID (create share link) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
145	Share-link request observed in transit	Information disclosure	Medium	Mitigated		The request reveals which report is about to be shared, and the response carries the resulting secret URL.	HTTPS only with HSTS.

share URL (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
146	Token leaks through Referer header, browser history or logs	Information disclosure	High	Mitigated		The secret token sits in the URL, therefore anyone with log access thereby gains report access.	- Never log the token or full query strings, don't include share token in proxy logs, and keep the token lifetime short.
147	Recipient forwards the share link	Information disclosure	Medium	Open		A share token is a bearer credential, so anyone the recipient passes it to gains the same read access.	- Treat any token holder as anonymous and untrusted: the share view returns report content only.
148	Token enumeration	Information disclosure	High	Mitigated		This flow is the endpoint the candidate tokens are submitted against where attackers are guessing. See the identically titled threat on Validate Token and Show Report (E9).	See the identically titled threat on Validate Token and Show Report (E9).

share token (T3, unauthenticated) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
149	Share token observed in transit	Information disclosure	High	Mitigated		The token is a bearer credential carried in the URL, so anyone who observes the request gains the same read access as the recipient.	HTTPS only with HSTS
150	Token leaks through Referer header, browser history or logs	Information disclosure	High	Mitigated		The token travels in the URL on this request as well. See the identically titled threat on share URL for the full description.	See the identically titled threat on share URL.

report content (X2) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
151	Shared report cached by an intermediary	Information disclosure	High	Mitigated		The share page is unauthenticated, so a shared cache is more likely to store and re-serve it.	Don't cache on the share response, reinforced at the assumed reverse proxy (D3).

token generation request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
152	Unbounded token generation for a single report	Denial of service	Medium	Mitigated		Repeatedly requesting share links for the same report creates an unbounded set of simultaneously valid tokens and consumes storage in the token store.	Cap the number of active share links per report and per citizen, and expire or revoke a superseded token when a new one is issued for the same report.

share token + report ID (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
153	Share tokens stored in usable form	Information disclosure	High	Mitigated		This flow is what reaches the store, so is carrying the plaintext token. See the identically titled threat on Share Token Store (D6) for the full description.	See the identically titled threat on Share Token Store (D6).

report content (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
154	Closed report remains accessible	Information disclosure	High	Mitigated		This flow is the report content returned after that check should have run. See the identically titled threat on Validate Token and Show Report (E9) for the full description.	See the identically titled threat on Validate Token and Show Report (E9).

token and report association (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
155	Closed report remains accessible	Information disclosure	High	Mitigated		This flow must carry the report's current closed state so E9 can check it at read time. See the identically titled threat on Validate Token and Show Report (E9).	See the identically titled threat on Validate Token and Show Report (E9).

requested report ID (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
156	Report identifier substituted after token validation	Information disclosure	High	Mitigated		If the report ID comes from the request rather than the token record, a valid token can be pointed at any report.	Resolve the report identifier exclusively from the stored token association, ignore any report ID in the request.

token lookup (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
157	Candidate token injected into the lookup query	Tampering	High	Mitigated		The candidate token is supplied by an unauthenticated caller and may contain SQL injection code.	Use paramaterisation to prevent SQL injection.

Proposed Changes

Owner:
Reviewer:
Contributors:
report.releasedAt: Wed Aug 12 2026

Executive Summary

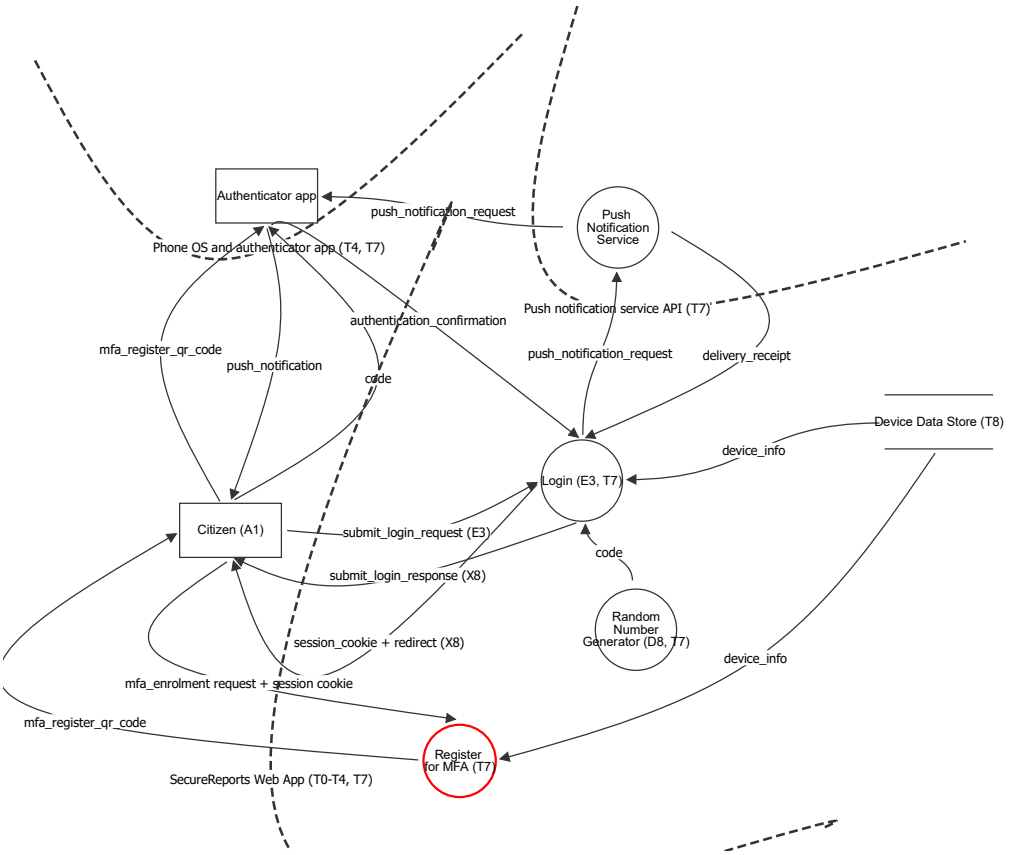
High level system description

Not provided

Summary

Total Threats	182
Total Mitigated	145
Total Accepted	2
Total Open	35
Open / Critical Severity	0
Open / High Severity	1
Open / Medium Severity	0
Open / Low Severity	0
Open / TBD Severity	34

Multiple Factor Authentication



Multiple Factor Authentication

Citizen (A1) (Actor)

Description: A Citizen is a user of the system who submits reports to be read and responded to by police officers

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Citizen doesn't own email address	Spoofing	TBD	Mitigated		The user registers with an email they don't own.	- Use email verification code after registration.
2	Bad actor logs in using stolen credentials	Spoofing	TBD	Mitigated		Bad actor finds a user's email and password (through measures like phishing emails etc) and attempts to use it to log in.	- Enforce multi-factor authentication to ensure that stolen credentials can't allow a user to log in with credentials alone.
3	User's session cookie is stolen	Spoofing	TBD	Mitigated		A bad actor obtains a session cookie belonging to a legitimate authenticated user, allowing them to impersonate a citizen by without knowing their password.	- Short session expiry or cookie-token lifetime - Session id rotation after privilege changes or re-authentication - Require re-authentication for sensitive actions - Use session binding to reject or flag requests where the session is used from a different context
61	Weak Password	Spoofing	TBD	Mitigated		Passwords can be guessed if they are too weak, or have been reused on another application where a data breach occurred.	- Enforce strong password rules such as 10 characters and at least one of capital letter, number, special character, or pass-phrases - Limit the number of times a user can attempt to login (so that bad actors have less chances at guessing correctly)

Login (E3, T7) (Process)

Description: The action to log a citizen into the SecureReports app

Number	Title	Type	Severity	Status	Score	Description	Mitigations
12	Flooding the process with requests	Denial of service	TBD	Mitigated		See #4	
13	Process replaced by scammer	Spoofing	TBD	Accepted		See #5	
14	Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		See #7	
15	Process has been tampered with	Tampering	TBD	Mitigated		See #6	
16	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		See #8	

Number	Title	Type	Severity	Status	Score	Description	Mitigations
17	Injection attack - tampering	Tampering	TBD	Mitigated		See #9	
18	Injection attack - disclosure	Information disclosure	TBD	Mitigated		See #10	
19	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		See #11	
20	Spoofed login session	Spoofing	TBD	Mitigated		Stolen cookies remain valid indefinitely, letting an attacker impersonate the citizen long after the cookie was stolen	- Have an idle-timeout and absolute-timeout for all session IDs. - Invalidate a session ID once logout occurs.
21	Non secure session cookie sent back to the Citizen	Spoofing	TBD	Mitigated		If the response from the login process fails to set the HttpOnly flag on the session cookie, a malicious script running on the front end could read/write to the cookie and allow a Bad Actor to impersonate the citizen.	- Use the HttpOnly + Secure flag on cookies to prevent client side javascript code from reading to the cookie. This also prevents writing in most browsers.
22	Account enumeration via error messages	Information disclosure	TBD	Mitigated		The application returns different responses or takes a different amount of time to respond depending on whether an email address is registered, has the wrong password, or hasn't been verified yet. This information can be used to determine which email addresses have accounts on the system.	- Return the same comprehensive error message for all failure cases. - Add timing delays so that timing differences are not observable.

Register for MFA (T7) (Process)

Description: This process occurs after email verification and first-time log in (T4)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	Flooding the process with requests	Denial of service	TBD	Mitigated		A bad actor could send many requests to the server because it is a process that does not require authentication.	- Rate limiting for requests from the same IP on the reverse proxy. - Add load balancing to reverse proxy to spread request traffic across servers.
5	Process replaced by scammer	Spoofing	TBD	Mitigated		The official registration page has been replaced by another malicious page users are redirected to from scam emails, or text messages, or links posted online. Specific to Register Process: This can mean Bad Actor can get access to the QR code used to enrol in on the authenticator app, allowing them to set the authenticator app up on the user's behalf.	- Buy similar domain names with typos. Specific to Register MFA process: - Bind enrolment to an existing authenticated session. - Notify the citizen's pre-existing verified channel on any new enrolment, with one-click revoke that collapses the attacker's window even if the QR is captured.
6	Process has been tampered with	Tampering	TBD	Mitigated		The process has been altered by an external malicious actor. This can be done by modifying source code and binaries (static tampering), injecting a different process (DLL injection), or memory corruption that can disrupt execution flow (e.g. corrupting heap metadata like function pointers).	- Monitor and keep a record of process ID to identify process restart. - Add fingerprinting to process, e.g., change time, binary / script hash. - Monitor process in/out and flag unexpected data patterns, malformed payloads unusual destinations or processes that deviate from usual register behaviour.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
7	Process leaks unsafe data via memory	Information disclosure	TBD	Mitigated		Process logs sensitive data, or stores data in unsafe (e.g., shared) memory	- Use type-safe and memory-safe programming language e.g. Java, Rust - Ensure log entries don't contain sensitive data. - Monitor process usages and in/out for suspicious activity.
8	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		Process runs at a higher priority level (e.g., permissions, resources), or accesses to assets outside its permission level.	- Apply principle of least privilege for process and used resources. - Run process on the OS with as few permissions as required, using a dedicated technical account rather than a normal user account with interactive and logon permissions. - Run in container with dropped capabilities.
9	Injection attack - tampering	Tampering	TBD	Mitigated		Malicious data is passed to the process to cause harm to the underlying data, e.g., override data or inject malicious data.	- Decode, normalise, and canonicalise email and password before validation and storage - Sanitise input for script-like requests, e.g., javascript, SQL, etc. - Enforce strict Content Security Policy to prevent harmful scripts to be transmitted (e.g., js, py, sh files). - Add parameterised queries/ prepared statements for all DB access.
10	Injection attack - disclosure	Information disclosure	TBD	Mitigated		Malicious data is passed to the process to gain access to or exfiltrate data stored in database.	- Decode, normalise, and canonicalise email and password before validation and storage - Sanitise input from script-like statements, e.g., javascript, SQL, etc. - Enforce strict CSP policy to prevent harmful scripts to be transmitted (e.g., js, py, sh files).
11	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		Logs may leak sensitive data such as email, password, IP, and session IDs, when tracing user actions.	- Ensure that all sensitive PII data other than user ID are not logged. - Conduct regular log audits.
62	MFA enrolment secret exposed via QR code capture	Information disclosure	TBD	Open		The QR code encodes the shared secret used to generate future authentication codes/approvals. If a Bad Actor captures the QR code (e.g. screen-sharing, malware taking a screenshot), they can enrol their own device against the citizen's account.	- Display the QR code only briefly and only after re-authentication. - Warn the citizen not to share their screen while the QR code is displayed.

code (Data Flow)

Description: 2-digit code to be used for push notification verification

Number	Title	Type	Severity	Status	Score	Description	Mitigations

code (Data Flow)

Description: User (A1) types in the 2-digit code into a input box on the authenticator app, after being prompted by notification

Number	Title	Type	Severity	Status	Score	Description	Mitigations
39	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #14	
40	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		See #17	
41	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the login credentials that the citizen supplied and log them into an account that is not theirs. Subsequent actions such as report making will then be performed on the attackers account.	- Use HTTPS (or another encrypted protocol) to send data over the network

delivery_receipt (Data Flow)

Description: Acknowledgement (or error) that the request for a push notification has been sent to the device

Number	Title	Type	Severity	Status	Score	Description	Mitigations
50	Push notification channel flooded	Denial of service	TBD	Mitigated		See #56	

push_notification_request (Data Flow)

Description: An API request for a push notification to be sent, including the device token that the notification should be sent to.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
47	Push authentication request tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor intercepts and alters the push notification request (e.g. swapping the device token) so that the prompt is sent to an attacker-controlled device instead of the citizen's, or is silently dropped.	- Use HTTPS (or another encrypted, integrity-protected protocol) to send data over the network. - Validate that the device token belongs to the account initiating the login before dispatching.
48	Push notification channel flooded, delaying delivery	Denial of service	TBD	Mitigated		See #56	

authentication_confirmation (Data Flow)

Description: Authenticator app sends a payload to the login process to confirm that the user is approved and can be logged in (T4).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
51	Push authentication request tampered with in transit	Tampering	TBD	Mitigated		See #47	- Cryptographically sign payload and ensure the Login process verifies this signature to catch tampering in transit.

push_notification_request (Data Flow)

Description: Request for a push notification reaches the operating system of the phone via a TLS connection (D2) that is always open (not modelled here for simplicity). The OS then hands this request to the authenticator app to notify the person

Number	Title	Type	Severity	Status	Score	Description	Mitigations
49	Push authentication request tampered with in transit	Tampering	TBD	Mitigated		Payload travels over an OS-managed push notification channel between the citizen's device and the push notifications service, rather than a connection the application itself. If a connection terminates while a payload is sent, and a malicious app or malware on the device can still intercept or spoof the notification.	- Use TLS encrypted channel to protect the payload from tampering in transit. - Keep the push notification request minimal so a party with visibility into the decrypted payload cannot meaningfully change what is displayed or trigger unintended actions. - Have the authenticator app fetch the actual request details (device, location, timestamp) in-app over its own authenticated channel rather than trusting content from the request from the OS.

session_cookie + redirect (X8) (Data Flow)

Description: Once authenticated (T4), the user will receive their session cookie (AS3) and be redirected to their main page (based on their privileges).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
54	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		See #58	
55	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		See #59	

submit_login_response (X8) (Data Flow)

Description: Includes the 2-digit code to be displayed on the user's screen (A1)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #14	
60	MFA code altered in transit	Tampering	TBD	Mitigated		If a Bad Actor observed and altered the response from the login process to a different code that is not correct, preventing the user from being able to authenticate their log in.	- Use HTTPS (or another encrypted protocol) to send data over the network

push_notification (Data Flow)

Description: App sends push notification to the user's (A1) phone, indicating the user needs to authenticate a sign in

Number	Title	Type	Severity	Status	Score	Description	Mitigations
52	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #56	

Number	Title	Type	Severity	Status	Score	Description	Mitigations
53	Push notification tampered with in transit	Tampering	TBD	Mitigated		See #59 Push notification could be altered to redirect you to a malicious site on click.	

mfa_register_qr_code (Data Flow)

Description: Citizen (A1) scans QR code displayed on their screen using their authenticator app. This adds the website to their authenticator app.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Flooding the network with traffic	Tampering	TBD	Mitigated		See #56	
67	QR code disclosed to a bad actor	Information disclosure	TBD	Mitigated		See #58	
68	QR code tampered with in transit	Tampering	TBD	Mitigated		See #59	

device_info (Data Flow)

Description: Information such as device_id, associated user_id, device_token for push notification, date enrolled etc.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
33	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #14	
35	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		See #17	
37	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the login credentials that the citizen supplied and log them into an account that is not theirs. Subsequent actions such as report making will then be performed on the attackers account.	- Use HTTPS (or another encrypted protocol) to send data over the network

mfa_enrolment request + session cookie (Data Flow)

Description: From the webpage, a citizen (A1) requests a QR code to register for MFA on their authenticator app.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Flooding the network with traffic	Tampering	TBD	Mitigated		See #56	

Authenticator app (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
42	MFA push-notification fatigue	Spoofing	TBD	Mitigated		A Bad Actor who already has the citizen's password repeatedly triggers login attempts, sending a flood of push approval prompts to the citizen's authenticator app in the hope that the citizen approves one out of confusion, annoyance, or accident, allowing the Bad Actor to be authenticated.	- Require the citizen to enter a number shown on the login screen into the app (number matching) rather than a simple approve/deny tap. - Rate limit the number of push prompts sent for a single account within a time window. - Show contextual information in the prompt (approximate location, device, time) to help the citizen recognise unfamiliar requests. - Temporarily lock the account or require step-up verification after repeated unactioned/denied prompts.
43	Citizen denies approving a fraudulent authentication	Repudiation	TBD	Mitigated		A citizen who approved a push prompt, intentionally or by mistake, later denies having done so, leaving no way to determine whether the approval was legitimate or the account was compromised.	- Log every push prompt sent and every approval/denial decision, including timestamp, device identifier, and approximate location. - Notify the citizen via email whenever a new device is approved or an approval occurs from an unfamiliar location.

Random Number Generator (D8, T7) (Process)

Description: The dependency process that generates randoms to be used for generating session ids, verification code, and seeds.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
23	Library replaced with a fake/ malicious library	Spoofing	TBD	Mitigated		Legitimate library replaced with a fake library by an actor, so consuming applications unknowingly pull numbers from an attacker controlled source.	- Code signing - require the library to be crypto-graphically signed by a trusted authority, and verify this signature before load. - Add fingerprinting to process, e.g., change time, binary / script hash, and verify at load time or periodically at runtime. - Use fixed absolute library paths for loading. - Use an allow list to declare exactly which library version and path are permitted - Monitor unexpected restarts or new processes claiming to be the library.
24	Entropy/seed leakage	Information disclosure	TBD	Mitigated		Internal state or seed value of the random number generator is exposed, allowing a bad actor to reconstruct past or predict future random numbers.	- Choose a well audited library (and library version) with no reported vulnerabilities. - Monitor the 'health' of the entropy source
25	Generation algorithm makes random number guessable	Information disclosure	TBD	Mitigated		Random number generator uses an algorithm that is not cryptographically secure, meaning even without the internal state being known, a past and future output can be guessed using brute force.	- Prefer libraries that use algorithms specified in NIST SP 800-90A/B/C or equivalent, which have been formally analyzed for prediction resistance. - Ensure that library uses an algorithm such that compromise of current state does not allow reconstruction of past outputs or prediction of future outputs.
26	Process has been tampered with	Tampering	TBD	Mitigated		See #24	
27	Injection attack - tampering	Tampering	TBD	Mitigated		See #26	- Choose a well audited library (and library version) with no reported vulnerabilities.

Push Notification Service (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
44	Push Notification Service replaced with malicious service	Spoofing	TBD	Mitigated		A Bad Actor stands up a malicious endpoint that mimics the Push Notification Service, causing the Login process to send authentication push requests to an attacker-controlled service instead of the legitimate one.	- Use mutually authenticated TLS between the Login process and the Push Notification Service. - Pin or verify the provider's certificate/ public key. - Use signed API credentials issued by the provider and rotate them regularly.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
45	Authentication metadata disclosed to third-party provider	Information disclosure	TBD	Mitigated		Push notifications are typically relayed through a third-party provider (e.g. Apple APNs, Google FCM) outside the application's direct control. This can expose login context such as device token, approximate location, or timing of sign-in attempts to that third party.	- Keep the notification payload minimal - Review the data-handling agreement/ terms of the push provider.
46	Push Notification Service unavailable or flooded	Denial of service	TBD	Mitigated		A Bad Actor floods the Push Notification Service with requests, or the third-party provider experiences an outage, preventing authentication prompts from being delivered and locking citizens out of login.	- Rate limit push requests per account and per source IP. - Monitor provider status and delivery success rate, and alert on anomalies. - Choose a trusted push notification service.

Device Data Store (T8) (Store)

Description: Stores information about the devices that use the application, such as device_id, associated user_id, device_token for push notification, date enrolled etc.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
28	Exfiltrate data	Information disclosure	TBD	Mitigated		An unregistered or unprivileged process or user tries to get access to the datastore.	- Use strong credentials for database access stored in secured password vaults. - Credentials are passed at runtime from environment variables for most DB owners / users with least privilege principle. - Disable default DB user. - Separate between read and read/write permissions. - Monitor for unusual activity, e.g., bulk data accessed, unusual number of simultaneous requests or connections. NOTE: Copied straight from example
29	DB is accessed and tampered with	Tampering	TBD	Mitigated		A bad actor or process tamper the datastore directly.	- Allow full access to datastore to only a restricted set of personnel. - Store strong credentials in password vaults. - Monitor for and log all direct command line accesses to the datastore. - Keep regular backups in a separate location.
30	DB is unreachable	Denial of service	TBD	Mitigated		A bad actor floods the datastore with requests and overloads the database connection pool, preventing access to the database.	- Monitor suspicious requests and disallow requests from abusing local IPs or ban abusing user. - Allow for replication and deployment with limited downtime when the load increases slightly. - Use a pool of connection with query timeouts to release resources fast even for unsuccessful queries. - Set a maximum number of connections allowed, with a queue. - Set a limit to the memory used by each query. - Monitor disk and CPU usage.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
31	Citizen PII stored unencrypted	Information disclosure	TBD	Mitigated		If the database disk is unencrypted and obtained by a Bad Actor, the data can be read directly.	- Encrypt the database at rest. - Keep the database isolated to the app host - don't serve access to the database over the network
32	No audit trail of account data changes	Repudiation	TBD	Mitigated		Without an audit log, a citizen could deny changes they have made to their account credentials. There is also no way of tracking when and whether a Bad Actor has submitted changes to a citizens account.	- Maintain an audit log of account change events such as registration and verification. - Include timestamps in logs - Monitor logs for abnormalities

submit_login_request (E3) (Data Flow)

Description: Sends request to log in to the application by providing their email (if they are a regular citizen (A1)) or username (if they are police (A3)) and their password (AS1).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
56	Flooding the network with traffic	Denial of service	TBD	Mitigated		A bad actor floods the request channel with high-volume traffic, exhausting network/connection capacity and blocking legitimate requests from reaching the server.	- Rate limiting for requests from the same IP.
58	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		Email address and password is accessed by a bad actor on the network, allowing them to steal the user's credentials.	- Use HTTPS (or another encrypted protocol) to send data over the network - Use verification code to ensure stolen credentials cannot be used to log in.
59	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the login credentials that the citizen supplied and log them into an account that is not theirs. Subsequent actions such as report making will then be performed on the attackers account.	- Use HTTPS (or another encrypted protocol) to send data over the network

device_info (Data Flow)

Description: Information such as device_id, associated user_id, device_token for push notification, date enrolled etc.

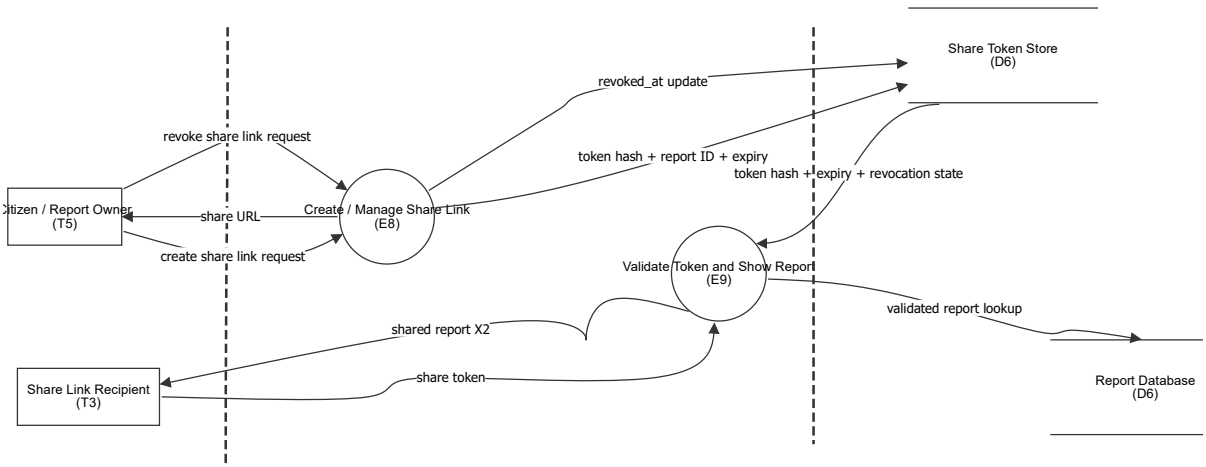
Number	Title	Type	Severity	Status	Score	Description	Mitigations
34	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #14	
36	Citizen information disclosed to a Bad Actor	Information disclosure	TBD	Mitigated		See #17	
38	Citizen login data tampered with in transit	Tampering	TBD	Mitigated		A Bad Actor could tamper with the login credentials that the citizen supplied and log them into an account that is not theirs. Subsequent actions such as report making will then be performed on the attackers account.	- Use HTTPS (or another encrypted protocol) to send data over the network

mfa_register_qr_code (Data Flow)

Description: Register process shows the user (A1) a QR code for them to scan using their authenticator app.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	Flooding the network with traffic	Denial of service	TBD	Mitigated		See #56	
64	MFA secret QR code tampered with in transit	Tampering	TBD	Mitigated		See #59	
65	QR code disclosed to a bad actior	Tampering	TBD	Mitigated		See #58	

Reports Sharing Expiry and Revocation



Reports Sharing Expiry and Revocation

Citizen / Report Owner (T5) (Actor)

Description: Authenticated report owner who can create and revoke share links for their own reports.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Share Link Recipient (T3) (Actor)

Description: Unauthenticated user who possesses a valid share token and is permitted to view the associated report.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Create / Manage Share Link (E8) (Process)

Description: E8 Share link creation and revocation. Creates a cryptographically random token for a report owned by the citizen, assigns an expiry time, stores the token association and allows the owner to revoke the token.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
201	Unauthorised revocation of another user's share link	Elevation of privilege	High	Mitigated		A citizen could attempt to revoke a share token belonging to a report they do not own by manipulating the token or report identifier in the revocation request.	Require authentication and perform a server-side ownership check before changing the revocation state of a share token. The authenticated user must own the associated report.
134	Creating a link for someone else's report	Elevation of privilege	High	Mitigated		A citizen submits another citizen's report ID to E8 and obtains a working share link for a report they do not own.	Server side ownership check before creating the token.

Share Token Store (D6) (Store)

Description: Stores hashed share tokens, associated report IDs, creation time, expiry time and revocation state. Plaintext token is returned only when the link is created and is not stored in usable form.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
205	Share tokens stored in usable form	Information disclosure	High	Mitigated		If plaintext share tokens are stored, anyone obtaining read access to the token store obtains working share links.	Store only a cryptographic hash of each share token. The plaintext token should only be returned to the report owner when the link is created.

Validate Token and Show Report (E9) (Process)

Description: E9 Share link page. Receives a secret token and checks that it exists, has not expired and has not been revoked before retrieving and displaying the associated report.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
206	Expired or revoked token accepted	Information disclosure	High	Mitigated		If E9 does not correctly enforce expiration and revocation server-side, a leaked token could continue providing access after the owner intended the link to stop working.	On every request, validate token existence, compare the current time with expires_at, and reject tokens with a non-null revoked_at value before retrieving the report.
207	Token enumeration	Information disclosure	High	Mitigated		An attacker could repeatedly submit candidate tokens and use different responses to determine which tokens are valid.	Use high-entropy cryptographically random tokens, rate-limit token validation attempts and return generic responses for invalid, expired and revoked tokens.
138	Closed report remains accessible	Information disclosure	High	Mitigated		A share link created while the report was open keeps returning content after closure if report state is not re-checked during share-link authorisation.	Re-check the report state at read time and refuse the share view for closed reports.

Report Database (D6) (Store)

Description: Stores report information associated with the validated share token.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

revoke share link request (Data Flow)

Description: Authenticated report owner requests that an existing share token be revoked.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
210	Revocation request targets another report	Elevation of privilege	High	Mitigated		The client could modify the token or report identifier in the request to attempt to revoke a share link belonging to another citizen.	Use the authenticated session to identify the requester and verify ownership of the report associated with the token before revocation.

revoked_at update (Data Flow)

Description: E8 records the revocation time for the selected share token.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
212	Unauthorised revocation state change	Tampering	High	Mitigated		A token could be incorrectly marked revoked or unrevoked by an unauthorised request.	Allow only authenticated owners to initiate revocation and do not expose direct client control over the revoked_at value.

token hash + expiry + revocation state (Data Flow)

Description: E9 retrieves the token record to determine whether the token exists, has expired or has been revoked.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
215	Incomplete validity check	Information disclosure	High	Mitigated		If E9 checks the token but fails to check expiry or revocation state, a leaked token can continue to access the report after the owner intended access to end.	Require all three conditions before access: token exists, current time is before expires_at, and revoked_at is null.

create share link request (Data Flow)

Description: Authenticated report owner requests creation of a share link for their report.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

share URL (Data Flow)

Description: The newly generated share URL returned to the report owner.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
213	Leaked or forwarded share token	Information disclosure	High	Mitigated		A recipient can forward or disclose the share token, allowing anyone who obtains it to access the report while the token remains valid.	Use high-entropy tokens, enforce expiration, and allow the report owner to revoke the token.

token hash + report ID + expiry (Data Flow)

Description: E8 stores the hashed token together with its associated report and server-generated expiry time.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
211	Token metadata manipulated	Tampering	High	Mitigated		Expiry or report association could be modified if the application or database permissions allow unauthorised writes.	Use least-privilege database permissions and perform all token metadata changes through authorised server-side operations.

validated report lookup (Data Flow)

Description: E9 retrieves only the report associated with a successfully validated share token.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
216	Token-to-report association bypass	Elevation of privilege	High	Mitigated		A flaw could allow the requester to manipulate a report identifier and use a valid token to retrieve a different report.	Retrieve the report ID only from the validated token record and never accept a separate client-supplied report ID for the shared-report lookup.

share token (Data Flow)

Description: The secret token supplied by an unauthenticated recipient to access the shared report.

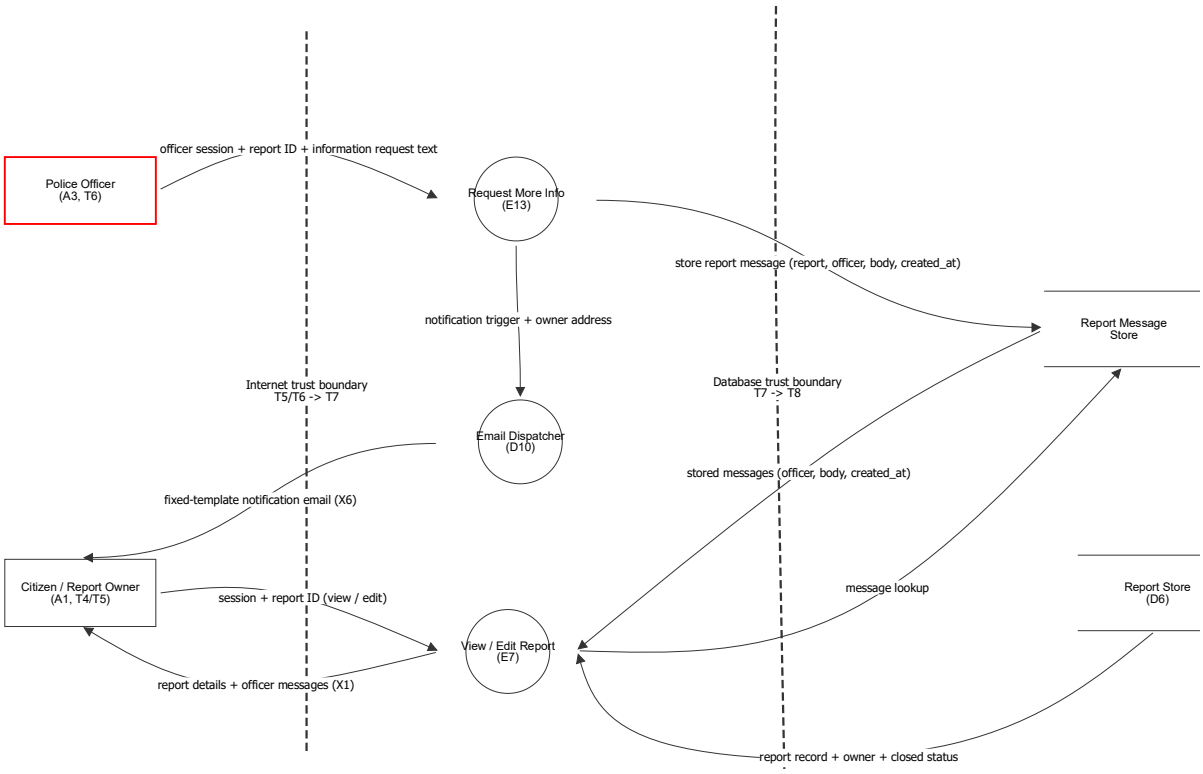
Number	Title	Type	Severity	Status	Score	Description	Mitigations
214	Token guessing or brute force	Spoofing	High	Mitigated		An attacker could submit guessed tokens in an attempt to obtain unauthorised access to reports.	Generate tokens using a cryptographically secure random number generator with sufficient entropy and rate-limit validation attempts.

shared report X2 (Data Flow)

Description: Report content returned to the share-link recipient only after token validation, expiry check and revocation check.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
217	Report disclosed after token should be invalid	Information disclosure	High	Mitigated		The shared report could be returned even after the token has expired or been revoked if E9 performs validation incorrectly or uses stale token state.	Perform validation immediately before retrieving and returning the report. Do not cache authorised report responses beyond the token validity period.

Request More Info via Stored Report Message



Request More Info via Stored Report Message

Police Officer (A3, T6) (Actor)

Description: A3 Police officer. T6 logged-in police officer: searches and views all reports, emails owners requesting more information and closes reports. Assumed unable to create or edit reports.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
158	Officer account compromised	Spoofing	High	Open		An attacker holding officer credentials, or an intercepted officer session, reads every report in the system and can close any of them.	Serve officer routes over HTTPS with HSTS, apply a short idle timeout and alert on unusual sign in locations.
159	Privileged officer action not recorded	Repudiation	High	Mitigated		There is no record of which officer performed a privileged action like closures or requesting further information	Write an audit record of officer ID, action, related text, timestamp, and protect the logs (X9) from the T6 role.

Citizen / Report Owner (A1, T4/T5) (Actor)

Description: A1 Citizen. T4 logged in citizen, T5 owner of the reports they created. Registers, submits reports with images, views and edits their own open reports and generates share links.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
160	Report owner's mailbox controlled by an attacker	Spoofing	Medium	Mitigated		An attacker who controls the report owner's mailbox receives the information request (X6) and the closure notification (X7)	Re-verify the address whenever it is changed, and keep incident detail out of email by linking back to the authenticated page.

Request More Info (E13) (Process)

Description: CHANGED. E13 Request more info. The officer writes the information request in the application and it is written to the Report Message Store as a ReportMessage (report, officer, body, created_at) rather than being sent. The outbound email is reduced to a fixed template containing no officer supplied content.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
166	Email header injection	Tampering	High	Mitigated		Injecting extra headers, letting the message be redirected or additional recipients added.	Include input validataion and construct headers through the mail library rather than by string building.
167	Information request sent to the wrong owner	Information disclosure	High	Mitigated		The recipient address is taken from the request instead of the stored report, so report detail reaches an attacker chosen mailbox.	Resolve the recipient from the report's stored owner in D6, never from client supplied input.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
168	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		Mail delivery to the citizen's external provider is outside the system's control so any incident detail ends up in an external mailbox in cleartext.	Keep incident detail out of the message body, send a short notice identifying the report by reference with a link back to the authenticated page
306	NEW - Officer supplied body stored without sanitisation	Tampering	TBD	Mitigated		The body is free text written by an officer and is later rendered on the report page. Script in the body would run in the report owner's browser.	- Escape the message body on output - Validate the body before presenting to user
307	NEW - Message written without recording the officer	Repudiation	TBD	Mitigated		If the officer field is not set from the authenticated session, then the police officer who wrote the message can not be identified	- Set officer and created_at server side from the session, never from the request.

Email Dispatcher (D10) (Process)

Description: CHANGED. D10 Email dispatcher. For information requests it now sends only a fixed template stating that there is an update on one of the citizen's reports and that the citizen should log in to view it. No officer supplied content and no links are included.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
172	Mail credentials disclosed	Information disclosure	High	Mitigated		Mail service credentials stored in the application configuration are read by an attacker, who can then send mail as the service.	- Keep configuration outside the web root, readable only by the T7 account, and rotate credentials. - Use secure secrets vault and apply the principle of least privilege.
173	Dispatcher abused to send arbitrary mail	Spoofing	High	Mitigated		An attacker who can reach the dispatcher sends convincing mail that appears to come from the service, enabling phishing against citizens.	- Authenticate which accounts have access to the mail service,
174	Email queue flooded	Denial of service	Medium	Mitigated		Repeated information requests or closures generate enough mail to exhaust the sending quota, delaying verification email (X5) and blocking registration.	- Rate limit outbound mail per officer and per report. - Monitor queue size for abnormalities
309	NEW - Notification confirms an address belongs to a report owner	Information disclosure	TBD	Mitigated		The notification tells anyone reading the mailbox that this address has an open report with the police.	- Do not name the report, the incident or the officer in the template.

View / Edit Report (E7) (Process)

Description: CHANGED. E7 View and edit report page. The report owner (T5) now also reads officer messages here and responds by editing the report as they do currently. Sets read_at on a message when it is displayed.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
100	Insecure direct object reference	Elevation of privilege	High	Mitigated		A user could change the report ID to access or edit another citizen's report if the system does not check who owns it.	- Derive the owner from the authenticated session and perform a server-side ownership check on the identifier for every report request.
101	Editing closed reports	Tampering	High	Mitigated		A crafted request could bypass a hidden edit button if the server does not enforce report state, so a closed report can still be modified.	- Always read the closed status with the record and reject updates server-side when the report is closed.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
102	Mass assignment of closed status	Elevation of privilege	High	Mitigated		A citizen may attempt to manipulate protected fields such as closed status.	- Allow-list editable fields and never accept closed status from the citizen.
103	Stored cross-site scripting in report description - Tampering	Tampering	High	Mitigated		A user could enter malicious script in a report description that runs when the report is viewed, and tampers with page content and behaviour.	- Safely encode the report description when displaying it so it is treated as text rather than code. - Use a strict CSP to prevent interaction with other domains - Use HttpOnly cookies so that a XSS attack cannot tamper with session ids and other important data.
104	Stored cross-site scripting in report description - Information Disclosure	Information disclosure	High	Mitigated		A user could enter malicious script in a report description that runs when the report is viewed, and tampers with page content and behaviour.	- Safely encode the report description when displaying it so it is treated as text rather than code. - Use a strict CSP to prevent interaction with other domains - Use HttpOnly cookies so that a XSS attack cannot tamper with session ids and other important data.
310	NEW - Messages shown to a citizen who does not own the report	Elevation of privilege	TBD	Mitigated		If the message can be found by report ID alone, any logged-in citizen (T4) who guesses a report ID reads the officer messages for it.	- Ensure the report and the report messages are only available to authenticated owner.

Report Store (D6) (Store)

Description: D6 Database. Asset: report content - date, time, location, description, created time, closed status and owner / created_by. Reachable only from the application (T7 -> T8). In this proposal it is also the source of the owner address used for the notification, so the address is never taken from the officer's request.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
175	Unauthorised report disclosure	Information disclosure	High	Mitigated		Direct database access could expose sensitive report information.	Restrict database access, use least privilege and protect data at rest.
176	Direct report tampering	Tampering	High	Mitigated		Database access could bypass application authorisation and modify reports.	Restrict direct database access and audit administrative changes.

Report Message Store (Store)

Description: NEW. Append only store holding ReportMessage records: report, officer, body, created_at and read_at. Because the message must be stored for the citizen to read it in the application, a record of which officer wrote what, and when, exists as a consequence of the change rather than as a separate mechanism alongside it.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
301	NEW - Officers delete report messages	Repudiation	TBD	Mitigated		An officer removes the record of an information request they sent, so there is no longer any evidence.	- Make the store append only - Do not grant officer roles delete permission on the message table
302	NEW - Bulk harvesting of police messages	Information disclosure	TBD	Mitigated		An unprivileged process or user reads the message store directly and obtains the body of every information request sent about every report	- Least privilege database credentials held only by the application - Monitor for bulk reads of the message table

Number	Title	Type	Severity	Status	Score	Description	Mitigations
303	NEW - Message store tampered with directly	Tampering	TBD	Mitigated		Direct database access is used to alter or insert a message, bypassing every application check on who may write one	- Restrict write access to the application identity (T7) only. - Keep the store append only so edits are not possible.
304	NEW - Message store unreachable	Denial of service	TBD	Mitigated		If the store is unavailable the officer cannot record a request and the citizen cannot read one, so the information request feature stops working entirely.	- Monitor availability and alert on failure. - Do not send a notification email if the message was not stored.

officer session + report ID + information request text (Data Flow)

Description: The officer's session, the report ID and the free text body of the information request. Unchanged by this proposal: the officer still writes the text, it is simply stored instead of emailed.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
179	Information request text observed in transit	Information disclosure	Medium	Mitigated		The text may quote incident detail from the report.	Use HTTPS protocol only with HSTS .

notification trigger + owner address (Data Flow)

Description: CHANGED. Only the recipient address and a template identifier are handed to the dispatcher. No officer supplied content crosses this flow any more.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
317	NEW - Officer content reaches the dispatcher	Tampering	TBD	Open		If any officer supplied field is still passed through to the dispatcher, phishing can still occur.	- Pass only the address and a fixed template ID. - Reject any free text on the dispatch path.

fixed-template notification email (X6) (Data Flow)

Description: CHANGED. X6. A fixed template saying only that there is an update on one of the citizen's reports and that they should log in to view it. Carries no officer supplied content and no links.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
184	Report content disclosed in cleartext email	Information disclosure	Medium	Mitigated		The information request email leaves the system on this flow. See the identically titled threat on Request More Info (E13).	See the identically titled threat on Request More Info (E13).
312	NEW - Fixed template cloned by an attacker	Spoofing	TBD	Open		Because the notification is identical every time, an attacker can clone it and add a malicious link, and the citizen has no way to tell the copy from the real one.	- Include no links in the template at all, so that any mail containing a link is visibly not from SecureReports.

session + report ID (view / edit) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
119	Insecure direct object reference	Tampering	High	Mitigated		This flow carries the client-supplied report ID that can be altered. See the identically titled threat on View / Edit Report (E7) for the full description.	See the identically titled threat on View / Edit Report (E7).
120	Edited report content observed in transit	Information disclosure	High	Mitigated		Both the retrieved report and the edited version cross the internet boundary in this exchange.	HTTPS only with HSTS (see model assumptions).

report details + officer messages (X1) (Data Flow)

Description: CHANGED. X1. The report page returned to the owner, now also carrying the officer messages held for that report.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
325	NEW - Message body rendered without escaping	Information disclosure	TBD	Open		The officer written body is rendered in the owner's browser. Unescaped output would run as script in the owner's session.	- Escape the body on output.

message lookup (Data Flow)

Description: NEW. Reads the messages held for the report the owner is viewing.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
318	NEW - Data or query altered in transit	Tampering	TBD	Open		The message lookup is altered between the application and the database.	- Encrypted database connection. - Database reachable only from the application host.
319	NEW - Data is read in transit	Information disclosure	TBD	Open		The message lookup is read on the internal network.	- Encrypted database connection.

store report message (report, officer, body, created_at) (Data Flow)

Description: NEW. The ReportMessage record written to the append only message store. This write, not a separate audit mechanism, is what creates the record of who asked for what.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
313	NEW - Data or query altered in transit	Tampering	TBD	Open		See #318	See #318
314	NEW - Data is read in transit	Information disclosure	TBD	Open		See #319	See #319

report record + owner + closed status (Data Flow)

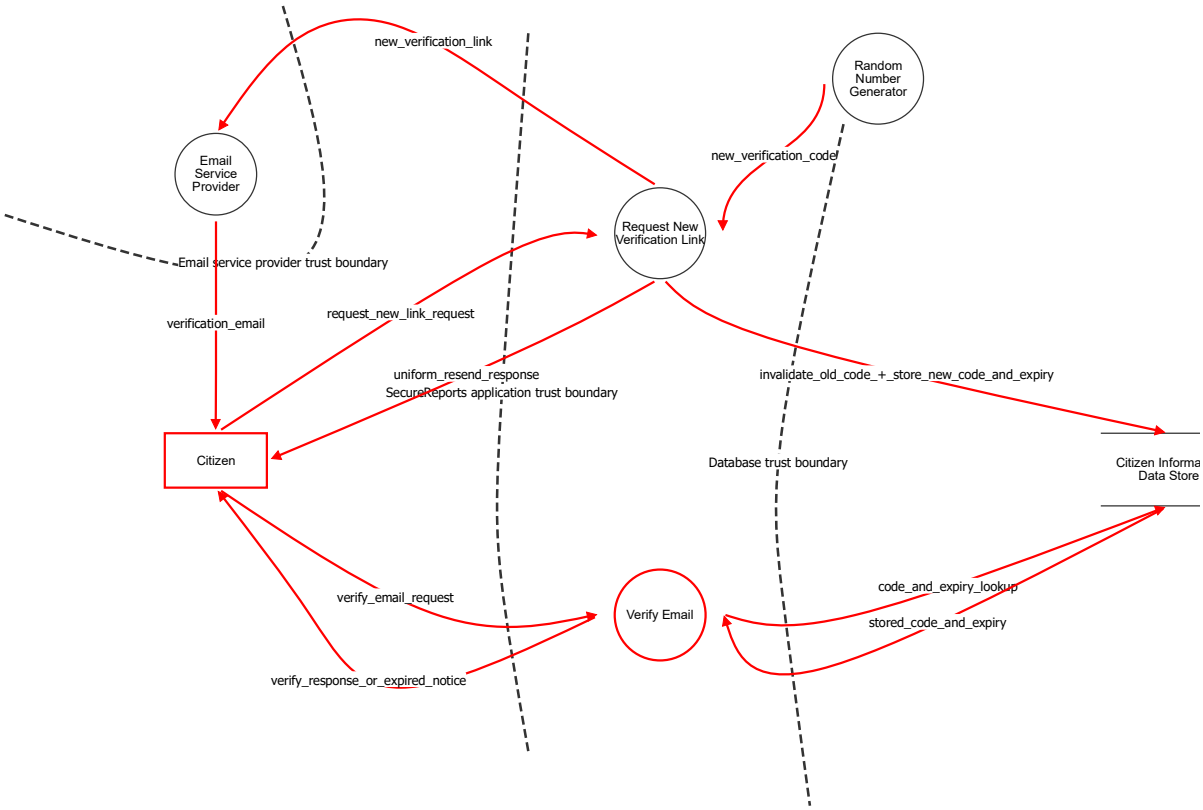
Number	Title	Type	Severity	Status	Score	Description	Mitigations
122	Editing closed reports	Tampering	Medium	Mitigated		This flow is where the closed status must be carried into the edit decision. See the identically titled threat on View / Edit Report (E7) for the full description.	See the identically titled threat on View / Edit Report (E7).

stored messages (officer, body, created_at) (Data Flow)

Description: NEW. The stored messages returned for display on the report page.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
321	NEW - Data or query altered in transit	Tampering	TBD	Open		See #318	See #318
322	NEW - Data is read in transit	Information disclosure	TBD	Open		See #319	See #319

Verification Code Expiry and Request New Link



Verification Code Expiry and Request New Link

Citizen (Actor)

Description: A citizen who has registered an account and must verify their email address before they can log in. In this proposal the citizen may also request a replacement verification link once the original has expired.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Weak password	Spoofing	TBD	Mitigated		Passwords can be guessed if they are too weak, or have been reused on another application where a data breach occurred.	- Enforce strong password rules such as 10 characters and at least one of capital letter, number, special character, or pass-phrases - Limit the number of times a user can attempt to login (so that bad actors have less chances at guessing correctly)
2	Citizen doesn't own email address	Spoofing	TBD	Mitigated		The user registers with an email they don't own.	- Use email verification code after registration. - (PROPOSED CHANGE) verification_code_expiry bounds the window in which a verification link for an address the registrant does not own remains usable to 10 minutes, so a mailbox compromised after that window no longer activates the account.
3	Bad actor logs in using stolen credentials	Spoofing	TBD	Open		Bad actors finds a user's email and password (through measures like phishing emails etc) and attempts to use it to log in.	
211	User's session cookie is stolen	Spoofing	TBD	Mitigated		A bad actor obtains a session cookie belonging to a legitimate authenticated user, allowing them to impersonate a citizen by without knowing their password.	- Short session expiry or cookie-token lifetime - Session id rotation after privilege changes or re-authentication - Require re-authentication for sensitive actions - Use session binding to reject or flag requests where the session is used from a different context

Email Service Provider (Process)

Description: Third-party email service that delivers the verification link to the citizen's mailbox. Outside the SecureReports trust boundary. The mail transport chain (MTA, mailbox) is collapsed into this element as it is unchanged by this proposal.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
228	Verification email spoofed as coming from SecureReports	Spoofing	TBD	Mitigated		The absence of SPF/DKIM/DMARC configuration allows Bad Actors to forge a verification email as coming from SecureReports.	- Configure SPF, DKIM, DMARC on the domain used to send verification emails - Monitor DMARC reports for unauthorised sending attempts
229	Email vendor becomes comprimised through tampering	Tampering	TBD	Mitigated		The email service and is assumed third-party infrastructure. If the vendor is compromised (e.g. its API keys are stolen), an attacker can send mail as SecureReports, or read mail in transit.	- Apply the principle of least privilege when storing API keys - Rotate keys regularly - Audit the third-party software's security - Store keys in a secret vault

Number	Title	Type	Severity	Status	Score	Description	Mitigations
233	Phishing SecureReports verify email clone	Spoofing	TBD	Accepted		A Bad Actor impersonates the legitimate SecureReports verification email with a link that directs citizens to a illegitimate site that harvests their email, password, and/or other personal data.	- Purchase all look-alike domains.
236	Verification token is retained in provider logs	Information disclosure	TBD	Mitigated		If the email provider logs message content/metadata, it can become exposed later if the email provider is compromised.	- Configure the provider's log retention period to a minimum - Conduct regular security audits of the third-party email provider - Invalidate the link once used by the citizen - (PROPOSED CHANGE) A token retained in provider logs is only usable for 10 minutes after issue, so the value of a later log compromise is bounded.
235	MTA becomes comprimised	Spoofing	TBD	Mitigated		The MTA is assumed third-party infrastructure. If the vendor is compromised (e.g. its API keys are stolen), an attacker can send mail as SecureReports.	- Apply the principle of least privilege when storing API keys - Rotate keys regularly - Audit the third-party software's security
237	Verification link read from a shared mailbox	Information disclosure	TBD	Mitigated		The verification email may be read by anyone with access to the citizen's mailbox. A Bad Actor may click on the link to also verify the citizen's account and gain information though the verification flow.	- Invalidate verification links once used by the citizen - (PROPOSED CHANGE) A link read from a shared mailbox is only usable within the 10 minute expiry window; after that the reader is refused and must request a new link, which is delivered to the same mailbox rather than to them.

Request New Verification Link (Process)

Description: NEW. Reached from the 'Didn't get your verification code?' button shown unconditionally on the login page. Takes an email address, invalidates any verification code already held for that account, stores a new code with verification_code_expiry set to 10 minutes from now, and sends a new verification link. The response is identical whether or not an account exists.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
268	NEW - Resend endpoint abused to exhaust email quota	Denial of service	TBD	Mitigated		An attacker may repeatedly request new verification codes, exhausting the email sending quota and delaying delivery of verification emails for other users. Because the request form is unauthenticated and visible on the login page, no account is needed to reach it.	- Rate limit the number of new verification codes that may be requested per email address, and per source IP address - Cap the total number of verification emails sent to a single address per timeframe
269	NEW - Previously issued verification codes remain valid after reissue	Spoofing	TBD	Mitigated		If issuing a new verification code does not invalidate the codes already issued for that account, every code ever sent stays usable.	- Delete all previously issued verification codes for the account whenever a new code is issued. - Enforce at the data layer that each account holds at most one valid verification code at a time.
270	NEW - Resend endpoint used to enumerate registered email addresses	Information disclosure	TBD	Mitigated		An attacker could submit candidate email addresses to the request-new-link endpoint and use differences in the response to learn which addresses already have an account.	- Return an identical response body and status code whether or not an account exists for the submitted address

Number	Title	Type	Severity	Status	Score	Description	Mitigations
255	NEW - Process replaced by scammer	Spoofing	TBD	Mitigated		See #178 (same threat on a different element in this diagram).	- As for #178.
256	NEW - Process has been tampered with	Tampering	TBD	Mitigated		See #164 (same threat on a different element in this diagram).	- As for #164.
257	NEW - Injection attack - tampering	Tampering	TBD	Mitigated		See #166 (same threat on a different element in this diagram).	- As for #166.
258	NEW - Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		See #181 (same threat on a different element in this diagram).	- As for #181.
259	NEW - Injection attack - disclosure	Information disclosure	TBD	Mitigated		See #182 (same threat on a different element in this diagram).	- As for #182.
260	NEW - Expose sensitive data in logs	Information disclosure	TBD	Mitigated		See #183 (same threat on a different element in this diagram).	- As for #183.
261	NEW - Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		See #184 (same threat on a different element in this diagram).	- As for #184.

Verify Email (Process)

Description: CHANGED. Redeems the verification code from the link. In this proposal it must check verification_code_expiry as well as the code itself: if the current server time is past the expiry, verification is refused and the citizen is shown a message telling them to request a new link.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
177	Flooding the verify email process with requests	Denial of service	TBD	Mitigated		A bad actor could send many requests to the server because it is a process that does not require authentication.	- Rate limiting for requests from the same IP on the reverse proxy. - Add load balancing to reverse proxy to spread request traffic across servers.
178	Verification process replaced by scammer	Spoofing	TBD	Open		The official registration page has been replaced by another malicious page users are redirected to from scam emails, or text messages, or links posted online.	- Buy similar domain names with typos.
179	Process has been tampered with	Tampering	TBD	Mitigated		See #164 (same threat on a different element in this diagram).	- As for #164.
180	Injection attack - tampering	Tampering	TBD	Mitigated		See #166 (same threat on a different element in this diagram).	- As for #166.
181	Process leaks data via unsafe memory	Information disclosure	TBD	Mitigated		Process logs sensitive data, or stores data in unsafe (e.g., shared) memory	- Use type-safe and memory-safe programming language e.g. Java, Rust - Ensure log entries don't contain sensitive data. - Monitor process usages and in/out for suspicious activity.
182	Injection attack - disclosure	Information disclosure	TBD	Mitigated		Malicious data is passed to the process to gain access to or exfiltrate data stored in database.	- Decode, normalise, and canonicalise email and password before validation and storage - Sanitise input from script-like statements, e.g., javascript, SQL, etc. - Enforce strict CSP policy to prevent harmful scripts to be transmitted (e.g., js, py, sh files).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
183	Expose sensitive data in logs	Information disclosure	TBD	Mitigated		Logs may leak sensitive data such as email, password, IP, and session IDs, when tracing user actions.	- Ensure that all sensitive PII data other than user ID are not logged. - Conduct regular log audits.
184	Process gets access to restricted resources	Elevation of privilege	TBD	Mitigated		Process runs at a higher priority level (e.g., permissions, resources), or accesses to assets outside its permission level.	- Apply principle of least privilege for process and used resources. - Run process on the OS with as few permissions as required, using a dedicated technical account rather than a normal user account with interactive and logon permissions. - Run in container with dropped capabilities.
271	NEW - Expiry field is stored but never evaluated at redemption	Spoofing	TBD	Mitigated		The verification_code_expiry field exists on the citizen record but the verification process does not evaluate it when the code is redeemed.	- Evaluate verification_code_expiry in the same server-side check as the verification code, and reject the request if the code has expired.

Random Number Generator (Process)

Description: Generates the verification code. Unchanged by this proposal, but included because the request-new-link flow needs a fresh code from it. Code entropy remains a primary defence against guessing; expiry only bounds the guessing window.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
161	Library replaced with a fake/malicious library	Spoofing	TBD	Mitigated		Legitimate library replaced with a fake library by an actor, so consuming applications unknowingly pull numbers from an attacker controlled source.	- Code signing - require the library to be crypto-graphically signed by a trusted authority, and verify this signature before load. - Add fingerprinting to process, e.g., change time, binary / script hash, and verify at load time or periodically at runtime. - Use fixed absolute library paths for loading. - Use an allow list to declare exactly which library version and path are permitted - Monitor unexpected restarts or new processes claiming to be the library.
162	Entropy/seed leakage	Information disclosure	TBD	Mitigated		Internal state or seed value of the random number generator is exposed, allowing a bad actor to reconstruct past or predict future random numbers.	- Choose a well audited library (and library version) with no reported vulnerabilities. - Monitor the 'health' of the entropy source
163	Generation algorithm makes random number guessable	Information disclosure	TBD	Mitigated		Random number generator uses an algorithm that is not cryptographically secure, meaning even without the internal state being known, a past and future output can be guessed using brute force.	- Prefer libraries that use algorithms specified in NIST SP 800-90A/B/C or equivalent, which have been formally analyzed for prediction resistance. - Ensure that library uses an algorithm such that compromise of current state does not allow reconstruction of past outputs or prediction of future outputs. - (PROPOSED CHANGE) Expiry bounds the time available to brute force a guessable code to 10 minutes. It complements, and does not replace, code entropy and rate limiting, which remain the primary defences against guessing.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
164	Process has been tampered with	Tampering	TBD	Mitigated		The process has been altered by an external malicious actor. This can be done by modifying source code and binaries (static tampering), injecting a different process (DLL injection), or memory corruption that can disrupt execution flow (e.g. corrupting heap metadata like function pointers).	- Monitor and keep a record of process ID to identify process restart. - Add fingerprinting to process, e.g., change time, binary / script hash. - Monitor process in/out and flag unexpected data patterns, malformed payloads unusual destinations or processes that deviate from usual register behaviour.
166	Injection attack - tampering	Tampering	TBD	Mitigated		Malicious data is passed to the process to cause harm to the underlying data, e.g., override data or inject malicious data.	- Choose a well audited library (and library version) with no reported vulnerabilities.

Citizen Information Data Store (Store)

Description: CHANGED. Holds the citizen record. Gains a verification_code_expiry field (a time, set to 10 minutes after the verification code is issued) alongside the existing email address, hashed password, verification code and is_verified status.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
185	Exfiltrate data	Information disclosure	TBD	Mitigated		An unregistered or unprivileged process or user tries to get access to the datastore.	- Use strong credentials for database access stored in secured password vaults. - Credentials are passed at runtime from environment variables for most DB owners / users with least privilege principle. - Disable default DB user. - Separate between read and read/write permissions. - Monitor for unusual activity, e.g., bulk data accessed, unusual number of simultaneous requests or connections. NOTE: Copied straight from example
186	DB is accessed and tampered with	Tampering	TBD	Mitigated		A bad actor or process tamper the datastore directly.	- Allow full access to datastore to only a restricted set of personnel. - Store strong credentials in password vaults. - Monitor for and log all direct command line accesses to the datastore. - Keep regular backups in a separate location.
187	DB is unreachable	Denial of service	TBD	Mitigated		A bad actor floods the datastore with requests and overloads the database connection pool, preventing access to the database.	- Monitor suspicious requests and disallow requests from abusing local IPs or ban abusing user. - Allow for replication and deployment with limited downtime when the load increases slightly. - Use a pool of connection with query timeouts to release resources fast even for unsuccessful queries. - Set a maximum number of connections allowed, with a queue. - Set a limit to the memory used by each query. - Monitor disk and CPU usage.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
247	Citizen PII stored unencrypted	Information disclosure	TBD	Mitigated		If the database disk is unencrypted and obtained by a Bad Actor, the data can be read directly.	- Encrypt the database at rest. - Keep the database isolated to the app host - don't serve access to the database over the network
248	No audit trail of account data changes	Repudiation	TBD	Mitigated		Without an audit log, a citizen could deny changes they have made to their account credentials. There is also no way of tracking when and whether a Bad Actor has submitted changes to a citizens account.	- Maintain an audit log of account change events such as registration and verification. - Include timestamps in logs - Monitor logs for abnormalities - (PROPOSED CHANGE) Issue, reissue and expiry of verification codes are new events that must appear in the same audit log.

request_new_link_request (Data Flow)

Description: Email address submitted from the 'Didn't get your verification code?' form. Unauthenticated.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
262	NEW - Flooding the network with traffic	Denial of service	TBD	Open		See #224 (same threat on a different element in this diagram).	- As for #224.
263	NEW - Request new link request is changed in transit	Tampering	TBD	Open		Request URL query parameters or body is tampered with by a bad actor.	- Use HTTPS so data is encrypted in transit
264	NEW - Submitted email address exposed to a bad actor	Information disclosure	TBD	Open		Email address and password is accessed by a bad actor on the network, allowing them to steal the user's credentials.	- Use HTTPS to send data over the network - Use verification code to ensure stolen credentials cannot be used to log in.

new_verification_code (Data Flow)

Description: Freshly generated verification code for the replacement link.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

invalidate_old_code_+_store_new_code_and_expiry (Data Flow)

Description: Deletes any verification code already held for the account and writes the new code together with verification_code_expiry set to 10 minutes from now.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
188	Data or query altered in transit	Tampering	TBD	Open		Bad actor alters query in transit (e.g. adding DROP tables to the query) or alters the data in the query to be incorrect (e.g. updating the query to set is_verified = true)	- Use local private network. - Use certificates to authenticate originating machine. - Use prepared statements on the caller side to minimise chances of effects of tampered data. - Use TLS/SSL encryption for all database connections.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
189	Data is read in transit	Information disclosure	TBD	Open		A bad actor captures the data in transit	- Use a secure private network - Use TLS/SSL encryption for all database connections.
190	Wire is overloaded by request	Denial of service	TBD	Open		The wire is overloaded by requests in the private network.	- Monitor network activity for suspicious patterns, e.g., corrupted processes - Drop suspicious packets and ban originating IPs (local firewalling)

new_verification_link (Data Flow)

Description: Replacement verification link handed to the email provider for delivery. Leaves the SecureReports trust boundary.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
249	Verification link altered in transit	Tampering	TBD	Open		A Bad Actor may observe and modify the verification link in transit, swapping the token to prevent a user from verifying their account, or altering the url so the citizen will be brought to a phishing page.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit
251	Request-new-link flood exhausts the email API	Denial of service	TBD	Open		Mass automated registrations can cause the Server's API limit to be reached or overload the email service provider	- Rate limit the registration request before the call to the ESP is performed. - Monitor API usage limits - Log traffic and audit logs regularly to detect anomalies. - (PROPOSED CHANGE) The request-new-link form is a second, unauthenticated path to the email API and must be rate limited in the same way as registration.

uniform_resend_response (Data Flow)

Description: Response shown to the submitter. Identical whether or not an account exists for the submitted address, so the form does not reveal which addresses are registered.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
20	Error messages reveal information about system state	Information disclosure	TBD	Open		An error message could contain information such as - server software & version - file system paths - stack traces - sensitive information such as email and password	- Don't have revealing error messages - Use HTTPS (or another encrypted protocol) to send data over the network. - (PROPOSED CHANGE) The request-new-link form must return an identical response whether or not an account exists, so this response in particular must reveal nothing about system state.
21	Request new link response is changed in transit	Tampering	TBD	Open		Response html/javascript is tampered with by a bad actor e.g., injecting a malicious script, altering the form's action URL to point to an attacker-controlled endpoint, or adding a hidden field.	- Use HTTPS (or another encrypted protocol) so data is encrypted in transit

verification_email (Data Flow)

Description: Email containing the new verification link, delivered to the citizen's mailbox.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
230	Verification link intercepted in transit via SMTP	Information disclosure	TBD	Open		If the email is not encrypted end to end, a Bad Actor could observe an unencrypted verification email.	- Use end-to-end encryption - Invalidate the link once used by the citizen - (PROPOSED CHANGE) An intercepted link is only usable within the 10 minute expiry window.
253	Verification link visible on screen	Information disclosure	TBD	Open		If the email displays the verification link on screen, it could be seen by a Bad Actor who then uses it to verify and exploit more information about the citizen's account through the verification flow.	- Obscure the verification link in the email as a hyperlinked button or styled link text rather than displaying the raw URL.

verify_email_request (Data Flow)

Description: Verification code presented by following the link.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
225	Flooding the network with traffic	Denial of service	TBD	Open		See #224 (same threat on a different element in this diagram).	- As for #224.
227	Citizen verify request is changed in transit	Tampering	TBD	Open		A Bad Actor could tamper with the verification code in the request to be invalid and prevent the Citizen from verifying their account.	- Use HTTPS (or another encrypted protocol) to send data over the network

code_and_expiry_lookup (Data Flow)

Description: Reads the stored verification code and its verification_code_expiry for the account.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
205	Data or query altered in transit	Tampering	TBD	Open		See #188 (same threat on a different element in this diagram).	- As for #188.
206	Data is read in transit	Information disclosure	TBD	Open		See #189 (same threat on a different element in this diagram).	- As for #189.
207	Wire is overloaded by request	Denial of service	TBD	Open		See #190 (same threat on a different element in this diagram).	- As for #190.

stored_code_and_expiry (Data Flow)

Description: Stored verification code and verification_code_expiry returned for server-side evaluation.

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Number	Title	Type	Severity	Status	Score	Description	Mitigations
265	NEW - Data or query altered in transit	Tampering	TBD	Open		See #188 (same threat on a different element in this diagram). The stored verification_code_expiry could be altered in transit so an expired code is evaluated as still valid.	- As for #188.
266	NEW - Data is read in transit	Information disclosure	TBD	Open		See #189 (same threat on a different element in this diagram).	- As for #189.
267	NEW - Wire is overloaded by request	Denial of service	TBD	Open		See #190 (same threat on a different element in this diagram).	- As for #190.

verify_response_or_expired_notice (Data Flow)

Description: On success, redirect to the login page. If the code has expired, verification is refused and the citizen is told to request a new link.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
224	Flooding the network with traffic	Denial of service	TBD	Open		A bad actor floods the request channel with high-volume traffic, exhausting network/connection capacity and blocking legitimate requests from reaching the server.	- Rate limiting for requests from the same IP.
226	Verify Email process response is changed during transit	Tampering	TBD	Open		See #21 (same threat on a different element in this diagram).	- As for #21.